
SAPHO

Manual do Usuário

Concepção, programação, simulação e visualização
de *soft-processors* com a IDE AURORA

NIPS-CERN — Núcleo de Instrumentação e Processamento de Sinais
Universidade Federal de Juiz de Fora (UFJF)

<https://nipscern.com/sapho>

Documento vivo — acompanha as versões da plataforma

Sumário

Prefácio	ix
I Primeiros passos	1
1 O que é o SAPHO	3
1.1 A plataforma em uma frase	3
1.2 O que é um soft-processor	3
1.3 As peças da plataforma	3
1.4 O fluxo de trabalho, de ponta a ponta	4
1.5 O exemplo que acompanha este manual	4
1.6 O laboratório por trás: NIPS-CERN	4
2 Instalação	5
2.1 Requisitos	5
2.2 Baixando e instalando	5
2.3 O que é instalado, e onde	5
2.4 Primeira execução	6
2.5 Desinstalação	6
3 Tour pela interface	7
3.1 A barra de título e a toolbar	7
3.2 O painel da árvore de arquivos	8
3.3 O editor	8
3.4 Os terminais	8
3.5 A barra de status	8
3.6 Paleta de comandos, notificações e tooltips	8
3.7 A tela de boas-vindas	9
II Projetos e processadores	11
4 Projetos	13
4.1 Criando um projeto	13
4.2 O arquivo .spf	13
4.3 Abrindo um projeto	13
4.4 Arquivos do projeto: quem é quem	14
4.5 As três visões da árvore	14
4.5.1 Visão Arquivos	14
4.5.2 Visão Hierarquia	14
4.5.3 Visão Pastas	14
4.6 Manutenção do projeto	15
5 O processador SAPHO	17
5.1 Visão geral da arquitetura	17
5.2 Os parâmetros fundamentais	17
5.3 O ponto flutuante do SAPHO	18
5.4 Como o programa vira circuito	18

5.5	Entrada, saída e o mundo externo	19
5.6	Um processador por tarefa	19
6	Criando processadores: o Hub de Processadores	21
6.1	Os campos do formulário	21
6.2	O que é gerado	22
6.3	Configurações de simulação do processador	22
6.4	Renomear e excluir processadores	23
III	A linguagem C±	25
7	C±: fundamentos	27
7.1	A estrutura de um programa	27
7.2	Diretivas: configurando o processador no próprio código	28
7.3	Constantes com <code>#define</code>	28
7.4	Comentários	29
7.5	Onde o arquivo vive	29
8	C±: tipos, operadores e controle	31
8.1	Tipos de dados	31
8.1.1	Literais complexos e o <code>i</code> reservado	31
8.1.2	Conversões entre tipos	31
8.2	Operadores	31
8.2.1	Operadores costum hardware	32
8.3	Estruturas de controle	32
8.4	Funções	32
8.5	Arrays	33
8.6	Variáveis e escopo	33
9	C±: entrada, saída e biblioteca padrão	35
9.1	Entrada e saída	35
9.2	Funções especiais	35
9.3	Funções não-lineares e de arredondamento	35
9.4	Funções para complexos	36
9.5	Álgebra linear em notação de Dirac	36
10	C±: recursos avançados	39
10.1	Interrupção: <code>#PRACA</code>	39
10.2	Sincronização com o hardware: <code>#TOAQUI</code>	39
10.3	FFT e o endereçamento bit-reverso	39
10.4	Quanto custa cada construção	39
10.5	O caminho C++	39
10.6	As mensagens do compilador	40
IV	O editor	41
11	O editor	43
11.1	Abas	43
11.2	Editor dividido	43
11.3	Realce de sintaxe	43
11.4	Diagnósticos e navegação (LSP)	43

11.5	Formatação automática	44
11.6	Busca no projeto	44
11.7	Widget de seleção e IA	44
11.8	Conflitos com o disco	44
11.9	Atalhos de teclado	44
V	Compilação e simulação	45
12	Compilação	47
12.1	O pipeline do YANC em três estágios	47
12.2	Os botões de compilação, um a um	47
12.3	Pré-requisitos e avisos	48
12.4	Onde cada artefato aparece	48
13	Os terminais	49
13.1	Quem escreve onde	49
13.2	Cartões, filtros e o modo verboso	49
13.3	Números de linha clicáveis	50
13.4	TCMD: o shell integrado	50
14	Simulação	51
14.1	Dois motores: Icarus Verilog e Verilator	51
14.2	Dois tipos de testbench: Verilog e cocotb	51
14.3	Parâmetros da simulação	51
14.4	Escolhendo os sinais das ondas	52
14.4.1	Quem manda no dump: a ordem de precedência	52
14.5	O seletor de <code>.gtkw</code>	52
14.6	Rodando	52
14.7	Teste de hardware (THTEST)	52
14.8	Simulando o <code>media_move1</code>	52
VI	Visualização	55
15	Formas de onda: GTKWave e Surfer	57
15.1	O que a AURORA prepara para você	57
15.2	Escolhendo o visualizador	57
15.3	GTKWave (fork NIPS-CERN)	58
15.4	Surfer	58
16	PRISM: o visualizador de RTL	61
16.1	Como abrir	61
16.2	Navegando no diagrama	61
16.3	Modo simulação (DigitalJS)	61
VII	Aurora Intelligence	63
17	Aurora Intelligence	65
17.1	Abrindo o painel	65
17.2	Configurando um provedor	65
17.2.1	CLIs por assinatura	65
17.3	O composer	66

17.4 Ferramentas: o que a assistente pode fazer	66
17.5 Conversas	66
17.6 Para que ela é boa (no fluxo SAPHO)	66
VIII Ferramentas de apoio	67
18 Dagr: controle de versão	69
18.1 Abrindo	69
18.2 A aba Changes	69
18.3 A aba History	69
18.4 Ramos e stashes	69
18.5 GitHub: publicar, clonar, sincronizar	70
19 Configurações	71
19.1 Geral	71
19.2 Aparência	71
19.3 Idioma	71
19.4 Terminal	71
19.5 Atalhos de Teclado	71
19.6 Assistente IA	71
19.7 Sobre	71
IX Manutenção da plataforma	73
20 Atualizações automáticas	75
20.1 Como funciona, do seu ponto de vista	75
20.2 A toolchain acompanha	75
20.3 Este manual também se atualiza	75
21 Solução de problemas	77
21.1 Compilação e simulação	77
21.2 Projeto e arquivos	77
21.3 Visualizadores	78
21.4 Aurora Intelligence	78
21.5 Onde estão os logs	78
X Apêndices	79
A Referência de diretivas	81
A.1 Diretivas de configuração	81
A.2 Diretivas comportamentais	81
A.3 Pré-processador	81
B Referência da biblioteca padrão	83
B.1 Entrada e saída	83
B.2 Funções especiais (economia de hardware)	83
B.3 Não-lineares (macros de assembly)	83
B.4 Complexos	83
B.5 Notação de Dirac	84
B.6 Famílias de instruções do processador	84

C	Atalhos de teclado	85
D	Glossário	87

Prefácio

Este livro é o manual de uso da plataforma SAPHO: da instalação ao primeiro processador funcionando em simulação, e de lá até os recursos mais avançados da IDE AURORA e da linguagem C $\pm$. Ele foi escrito para quem **nunca viu o SAPHO** — nenhum conhecimento prévio da plataforma é assumido, embora familiaridade básica com programação em C e com a ideia de circuitos digitais ajude.

Como este livro está organizado

O livro é dividido em módulos, um por componente da plataforma — cada componente do SAPHO é, na prática, um projeto por si só, e recebeu aqui a sua própria mini-documentação:

Parte I — Primeiros passos. O que é o SAPHO, instalação e um tour pela interface da AURORA.

Parte II — Projetos e processadores. O arquivo de projeto `.spf`, a arquitetura do processador SAPHO e o Hub de Processadores.

Parte III — A linguagem C $\pm$. A referência completa da linguagem: estrutura, tipos, operadores, E/S, biblioteca padrão e recursos avançados.

Parte IV — O editor. O editor Monaco e todo o apoio à edição: abas, divisão de painéis, diagnósticos, formatação, busca.

Parte V — Compilação e simulação. Os botões da toolbar, o pipeline do YANC, os seis terminais e os quatro caminhos de simulação.

Parte VI — Visualização. Formas de onda (GTKWave e Surfer) e o visualizador de RTL PRISM.

Parte VII — Aurora Intelligence. O assistente de IA integrado: configuração, provedores, ferramentas e permissões.

Parte VIII — Ferramentas de apoio. Dagr (controle de versão Git) e as configurações da IDE.

Parte IX — Manutenção. O atualizador automático e a solução de problemas.

Apêndices. Referências rápidas: diretivas, biblioteca padrão, atalhos de teclado e glossário.

Um exemplo único costura o livro: o processador `media_move1`, um filtro de média móvel construído do zero no projeto `MeuFiltro`. Ele nasce no Capítulo 6, ganha código na Parte III, compila na Parte V e aparece nas ondas e no RTL na Parte VI.

Convenções usadas neste livro

Botão Elementos clicáveis da interface: botões, itens de menu de contexto.

Ctrl+S Atalhos de teclado.

Nomes de comandos, diretivas, variáveis, valores literais.

arquivo.cmm Nomes de arquivos e caminhos.

Nome Ferramentas e componentes da plataforma.

As caixas coloridas seguem o vocabulário de cores da própria AURORA:

Nota (menta): dicas e informações que enriquecem o assunto ao lado.

- Informação** (ciano): contexto adicional, referências cruzadas, detalhes de bastidores.
- Aviso** (âmbar): restrições e comportamentos que costumam surpreender.
- Cuidado** (vermelho): ações destrutivas ou erros com consequências sérias.

Um documento vivo

Este manual acompanha o desenvolvimento da plataforma e é atualizado junto com a AURORA, o YANC e os demais componentes. Cada afirmação sobre comportamento foi extraída do código-fonte dos repositórios oficiais — não de documentos intermediários — na data da edição. Se você encontrar divergência entre o manual e o software, o software mudou primeiro: procure a edição mais recente ou abra uma issue nos repositórios do `nipscernlab`.

Parte I

Primeiros passos

O que é o SAPHO

1.1 A plataforma em uma frase

SAPHO (*Scalable-Architecture Processor for Hardware Optimization*) é uma plataforma completa para criar, programar, simular e visualizar *soft-processors*: você escreve um programa numa linguagem parecida com C, chamada $C\pm$ (lê-se “C mais-menos”), e a plataforma gera um processador dedicado em Verilog, sob medida para aquele programa, pronto para ser sintetizado em FPGA. Tudo acontece dentro de uma única aplicação de desktop, a IDE AURORA (*Advanced Utility Running Optimized Resource Architectures*), que roda localmente no Windows, sem depender de nuvem.

1.2 O que é um soft-processor

Um processador convencional (como o do seu computador) é um chip fixo: sua arquitetura foi decidida na fábrica e não muda. Um *soft-processor*, por outro lado, é um processador descrito em uma linguagem de descrição de hardware (HDL, no caso o Verilog) e implementado na malha reconfigurável de um FPGA. Isso significa que a arquitetura — largura da palavra, conjunto de instruções, quantidade de portas de entrada e saída, tamanho das memórias — é escolhida por você, e pode ser alterada e recompilada em minutos.

O SAPHO leva essa ideia ao extremo: o processador gerado contém *apenas* o hardware que o seu programa usa. Se o programa não usa divisão, o divisor não existe no circuito final. Se não usa ponto flutuante, nenhuma lógica de ponto flutuante é gerada. O resultado é um processador enxuto, previsível e eficiente, ideal para instrumentação científica e processamento de sinais em tempo real.

1.3 As peças da plataforma

O SAPHO é um guarda-chuva que reúne vários componentes. Cada um deles tem um módulo próprio neste manual.

Componente	O que é
AURORA	<i>Advanced Utility Running Optimized Resource Architectures</i> : a IDE de desktop — editor de código, gerenciador de projetos, botões de compilação e simulação, terminais, visualizadores. É o programa que você instala e abre.
YANC	<i>Yet Another Compiler</i> : a suíte de compiladores que traduz seu programa $C\pm$ (ou C++) no processador Verilog, nas imagens de memória e no <i>testbench</i> . Roda por baixo dos panos quando você clica nos botões de compilação.
Processador SAPHO	O <i>soft-processor</i> em si: o circuito Verilog parametrizável gerado pelo YANC (Capítulo 5).
<i>Toolchain bundle</i>	Um pacote portátil de ferramentas de EDA de código aberto (Icarus Verilog, Verilator, Yosys, GCC, Python + cocotb) que a AURORA baixa automaticamente na primeira execução.

Componente	O que é
PRISM	<i>Processor Rendering Interface for Schematic Models</i> : o visualizador de RTL embutido, que desenha o diagrama do circuito sintetizado, navegável módulo a módulo (Capítulo 16).
GTKWAVE / SURFER	Os dois visualizadores de formas de onda disponíveis para inspecionar simulações (Capítulo 15).
AURORA INTELLIGENCE	O assistente de IA integrado, que conversa sobre o projeto e age sobre a IDE por ferramentas curadas (Capítulo 17).

1.4 O fluxo de trabalho, de ponta a ponta

O caminho típico de quem usa o SAPHO é sempre o mesmo, e este manual o percorre na ordem:

1. **Criar um projeto** — uma pasta com um arquivo `.spf` que descreve tudo (Capítulo 4).
2. **Criar um processador** — escolher nome e parâmetros de arquitetura (Capítulo 6).
3. **Programar em C $\pm$** — escrever o comportamento no editor (Parte III).
4. **Compilar** — o YANC transforma o programa em Verilog + memórias + testbench (Capítulo 12).
5. **Simular** — Icarus Verilog ou Verilator executam o circuito (Capítulo 14).
6. **Visualizar** — formas de onda no GTKWave/Surfer, circuito no PRISM (Parte VI).
7. **Sintetizar no FPGA** — levar o `.v` e os `.mif` gerados para a ferramenta do fabricante (Quartus, Vivado etc.), fora da AURORA.

1.5 O exemplo que acompanha este manual

Para que cada conceito apareça em contexto, este manual constrói um processador de exemplo do zero e o acompanha até a forma de onda final: o `media_movel`, um filtro de média móvel que lê amostras de um sinal por uma porta de entrada, calcula a média das últimas amostras e escreve o resultado numa porta de saída. É um exemplo pequeno, mas exercita o essencial: entrada e saída, aritmética, laço principal e simulação com estímulo externo.

1.6 O laboratório por trás: NIPS-CERN

O SAPHO é desenvolvido e mantido pelo **NIPS-CERN**, núcleo de pesquisa da **Universidade Federal de Juiz de Fora (UFJF)**, em Minas Gerais. O laboratório colabora com o CERN no experimento **ATLAS** do LHC, onde técnicas de processamento de sinais em FPGA — o habitat natural do SAPHO — são aplicadas à instrumentação dos calorímetros. A plataforma também é usada no ensino, na disciplina de Dispositivos Lógicos Programáveis (DLP) da UFJF.

Todo o ecossistema é de código aberto, hospedado na organização `nipscernlab` no GitHub. O canal oficial de download para o usuário final é <https://nipscern.com/sapho>.

Este é um documento vivo: ele acompanha as versões da plataforma e é atualizado junto com a AURORA, o YANC e os demais componentes. Se algo na sua tela não corresponde ao manual, verifique se a sua instalação está atualizada (Capítulo 20).

Instalação

2.1 Requisitos

- **Windows 64 bits** (Windows 10 ou 11). A AURORA é, hoje, um aplicativo exclusivamente Windows.
- Cerca de 2 GB de espaço em disco (o aplicativo já traz embutida toda a cadeia de ferramentas de simulação e síntese).
- Nenhum outro programa é necessário: compiladores, simuladores e visualizadores vêm no pacote.

2.2 Baixando e instalando

1. Acesse <https://nipscern.com/sapho> e baixe o instalador — um arquivo chamado `sapho-aurora-Setup-vX.Y.Z.exe` (onde `X.Y.Z` é a versão).
2. Dê dois cliques no executável. O instalador é do tipo *one-click*: não há telas de opções nem escolha de pasta — ele instala, cria os atalhos na área de trabalho e no menu Iniciar e abre o aplicativo.

[Figura 1 — print pendente] *Página de download em nipscern.com/sapho*
Mostrar o botão de download do instalador.

[Figura 2 — print pendente] *Primeira abertura da AURORA: tela de splash e tela de boas-vindas*
Capturar a splash com a barra de progresso e, em seguida, a tela de boas-vindas com o fundo animado de aurora boreal.

Durante a instalação são registrados dois vínculos com o sistema:

- a extensão `.spf` (arquivo de projeto SAPHO) passa a abrir com a AURORA, com ícone próprio — dois cliques num `.spf` no Explorer abrem o projeto;
- o protocolo `sapho://` é registrado para integrações futuras via navegador.

O instalador ainda não é assinado digitalmente; o Windows SmartScreen pode exibir um aviso de “editor desconhecido” na primeira execução. Clique em **Mais informações** → **Executar assim mesmo**.

2.3 O que é instalado, e onde

Local	Conteúdo
Pasta do aplicativo	O executável da AURORA e a pasta <code>components/</code> com toda a toolchain: compiladores do YANC, simuladores (Icarus Verilog, Verilator), Yosys, GTKWave, Python com cocotb e os moldes de HDL do processador.
<code>%APPDATA%\textbackslashAurora-IDE\textbackslash</code>	Dados do usuário: logs (<code>logs/main.log</code>), lista de projetos recentes (<code>aurora-recents.json</code>), registro de versão para o pós-atualização.

Local	Conteúdo
Cofre de credenciais do Windows	Chaves de API da Aurora Intelligence e token do GitHub, cifrados pelo sistema operacional (DPAPI) — nunca em texto puro.

Os seus projetos ficam onde você quiser: a AURORA pergunta a pasta ao criar cada projeto (Capítulo 4).

2.4 Primeira execução

Ao abrir, a AURORA exibe uma tela de *splash* com o progresso real de inicialização e, em seguida, a **tela de boas-vindas**, com o fundo animado de aurora boreal, os botões **Novo Projeto** e **Abrir Projeto** e a lista de projetos **Recentes** (vazia na primeira vez). A partir daí, siga para o Capítulo 3 para conhecer a interface, ou direto para o Capítulo 4 para criar o primeiro projeto.

Cerca de seis segundos após abrir, a AURORA verifica silenciosamente se há uma versão mais nova; se houver, você verá a janela de atualização (Capítulo 20). Não é preciso fazer nada para manter o SAPHO em dia além de aceitar as atualizações quando oferecidas.

2.5 Desinstalação

Use Configurações do Windows → Aplicativos → Aurora-IDE → Desinstalar. Os projetos criados por você não são apagados — eles pertencem às pastas que você escolheu.

Tour pela interface

A janela da AURORA não tem a moldura padrão do Windows nem uma barra de menus Arquivo/Editar/Exibir: tudo vive numa **barra de título personalizada** (que concentra a toolbar), no **painel lateral** da árvore de arquivos, no **editor**, nos **terminais** e na **barra de status**. Este capítulo apresenta cada região; os detalhes de uso ficam nos capítulos dedicados.

[Figura 3 — print pendente] *Janela principal da AURORA com um projeto aberto*
Visão geral anotável: toolbar no topo, árvore à esquerda, editor ao centro, terminais embaixo, barra de status no rodapé.

3.1 A barra de título e a toolbar

A barra superior tem três zonas:

Zona esquerda — identidade e projeto:

- o ícone do SAPHO;
- **Alternar barra lateral** — mostra/esconde o painel da árvore de arquivos;
- **Novo Projeto** e **Abrir Projeto**.

Zona central — a cadeia do processador, na ordem em que você a usa:

- **Hub de Processadores** — cria processadores (Capítulo 6);
- **Compilar C±** (glifo “C±”) — compila o fonte C± até Verilog;
- **Configurações de simulação do processador** (engrenagem) — clock, número de ciclos e exibição de arrays;
- **Sintetizar Verilog** (pena) — verifica os arquivos Verilog com o Icarus Verilog;
- **PRISM** — abre o visualizador de RTL (Capítulo 16);
- **seletor de simulador** — Icarus Verilog ou Verilator (Capítulo 14);
- **seletor de visualizador** — GTKWave ou Surfer (Capítulo 15);
- **Analisar Verilog** (onda quadrada) — roda a simulação e abre as formas de onda;
- **Fast Sim** (“»”) — simulação rápida sem ondas (apenas com Verilator);
- **Cancelar** — interrompe todos os processos de compilação/simulação em andamento;
- **Configuração de ondas** — escolhe quais sinais entram no arquivo de ondas;
- **seletor de .gtkw** — escolhe o layout de ondas ativo;
- **Teste do processador sintetizado** (chip) — teste de E/S do processador via Verilator.

Zona direita — apoio e janela:

- **Controle de Versão** (Dagr, Capítulo 18);
- **Configurações do Aurora** (Capítulo 19);
- **Assistente IA** (estrela — Aurora Intelligence, Capítulo 17);
- os controles de janela: minimizar, maximizar/restaurar, fechar. Duplo clique na área vazia da barra também maximiza/restaura.

[Figura 4 — print pendente] *Toolbar completa em close*

Print da barra de título inteira, com um projeto aberto e os botões habilitados. Idealmente uma segunda versão com os tooltips de um dos botões visível.

3.2 O painel da árvore de arquivos

À esquerda fica o painel **ÁRVORE DE ARQUIVOS**, com o nome do projeto aberto ao lado do rótulo. Ele tem **três visões**, alternadas por um único botão no cabeçalho do painel:

1. **Arquivos** — a visão padrão, agrupada por processador, com os arquivos Verilog classificados automaticamente entre sintetizáveis e *testbench*;
2. **Hierarquia** — a árvore de instâncias de módulos do projeto, extraída pelo Yosys após uma síntese;
3. **Pastas** — um explorador de diretórios convencional, no estilo do VS Code, com criar/renomear/mover/excluir

O cabeçalho do painel traz ainda: **Novo arquivo**, **Atualizar**, **Buscar nos arquivos** (`Ctrl+Shift+F`), **Abrir no explorador do sistema**, **Backup do projeto** (gera um `.zip`), **Recolher tudo** e **Fechar projeto**. O uso completo das árvores está no Capítulo 4.

3.3 O editor

O centro da janela é o editor de código, baseado no MONACO (o mesmo motor do VS Code): abas múltiplas, divisão em até três painéis, realce de sintaxe para C $\pm$, assembly, Verilog e SystemVerilog, diagnósticos em tempo real e formatação automática. Tudo no Capítulo 11.

3.4 Os terminais

A área inferior reúne **seis terminais** em abas: **TCMM**, **TASM**, **TVERI**, **TWAVE** e **THTEST** são consoles de saída das etapas de compilação e simulação (cada etapa escreve no seu), e o **TCMD** é um terminal PowerShell interativo de verdade. Filtros por tipo de mensagem, exportação de log e números de linha clicáveis completam o conjunto (Capítulo 13).

3.5 A barra de status

No rodapé:

- à esquerda, o indicador **Pronto/Não Pronto** (o ponto verde/vermelho indica se o projeto está em condições de compilar) e o **processador ativo** (deduzido do arquivo `.cmm` em foco no editor);
- ao centro, o botão **Iniciar Compilação** (ícone de raio);
- à direita, avisos de estrutura (“Sem top-level”, “Sem testbench”), o motor de simulação selecionado, a posição do cursor (Ln, Col) e o indicador do GitHub/controle de versão.

3.6 Paleta de comandos, notificações e tooltips

- **Paleta de comandos** (`Ctrl+Shift+P`): busca difusa sobre os comandos da IDE, organizados em grupos (Compile, Project, View, Tools). Útil quando você sabe o nome do que quer fazer, mas não onde fica o botão.
- **Notificações**: eventos como criação de arquivos, troca de simulador e avisos de projeto aparecem como *toasts* no canto da janela.
- **Tooltips**: todo botão tem uma dica de ferramenta rica ao passar o mouse; elas podem ser desligadas nas Configurações.

3.7 A tela de boas-vindas

Sem projeto aberto, a AURORA mostra a tela de boas-vindas: a marca SAPHO com a *tagline* “Processador de Arquitetura Escalável para Otimização de Hardware”, a seção **Início** (**Novo Projeto**, **Abrir Projeto**) e a seção **Recentes** com os últimos dez projetos. O fundo é uma animação de aurora boreal.

Parte II

Projetos e processadores

Projetos

Tudo no SAPHO acontece dentro de um **projeto**: uma pasta no disco governada por um arquivo **.spf** (*SAPHO Project File*). O projeto reúne um ou mais processadores, os arquivos Verilog de integração (*top-level* e *testbench*) e os metadados que a AURORA usa para orquestrar compilação e simulação.

4.1 Criando um projeto

Clique em **Novo Projeto** (na toolbar ou na tela de boas-vindas). O modal **Criar Novo Projeto** pede:

Campo	Regras
Nome do Projeto	Apenas letras, números, sublinhado (<code>_</code>) e hífen (<code>-</code>). Sem espaços, acentos ou símbolos.
Local do Projeto	Somente leitura; preencha com o botão Procurar , que abre o seletor de pastas do Windows. A AURORA lembra o último local usado.

Clique em **Gerar Projeto**. Se algum campo violar as regras, o diálogo **Entrada Inválida** explica o problema (“Nome do Projeto e Local não podem conter espaços ou símbolos especiais. . .”).

[Figura 5 — **print pendente**] *Modal Criar Novo Projeto preenchido*
Com o nome `MeuFiltro` e um local qualquer, antes de clicar em Gerar Projeto.

No disco, é criada a estrutura `<Local>/<Nome>/<Nome>.spf`. Um projeto recém-criado ainda não tem processadores — a árvore abre praticamente vazia, e o próximo passo natural é o **Hub de Processadores** (Capítulo 6).

Para o exemplo condutor deste manual, crie o projeto **MeuFiltro**.

4.2 O arquivo .spf

O **.spf** é um arquivo JSON legível, com dois blocos:

- **metadata** — nome do projeto, datas de criação/modificação/última abertura, nome do computador e versão da AURORA que o criou;
- **structure** — a lista de **processors**, as pastas do projeto, o `topLevelFile`, o `testbenchFile` e as listas de arquivos Verilog sintetizáveis e de *testbench*.

Você raramente precisará editá-lo à mão: a AURORA é a única escritora do arquivo e o mantém em dia a cada ação na interface. Os caminhos são gravados relativos à pasta do projeto, então o projeto inteiro pode ser copiado, movido de máquina ou versionado no Git sem quebrar — ao abrir, a AURORA realinha os caminhos ao novo local.

4.3 Abrindo um projeto

Três caminhos:

- **Abrir Projeto** na toolbar ou na tela de boas-vindas, selecionando o `.spf`;
- dois cliques no `.spf` pelo Explorer do Windows (a extensão é associada na instalação);
- a lista de **Recentes** na tela de boas-vindas (até dez projetos, também disponíveis na *jump list* do ícone da AURORA na barra de tarefas do Windows).

Ao abrir, a AURORA carrega a árvore, restaura a lista de processadores e registra a data de abertura. Se o `.spf` tiver sumido do caminho registrado, o diálogo **Projeto Não Encontrado** avisa e o item sai da lista de recentes.

4.4 Arquivos do projeto: quem é quem

Um projeto maduro tem esta cara (exemplo com o nosso `MeuFiltro`):

```
MeuFiltro/
|-- MeuFiltro.spf          (o arquivo de projeto)
|-- media_movel/          (um processador)
|   |-- Software/media_movel.cmm
|   |-- Hardware/         (gerados: media_movel.v, *.mif)
|   |-- Simulation/       (gerado: media_movel_tb.v)
|-- TopLevel/             (Verilog de integracao, opcional)
`-- ...
```

Dois papéis especiais são atribuídos por você, pelo menu de contexto da árvore:

- **Top-level** — o módulo Verilog que integra o(s) processador(es); é o topo da síntese e o alvo do PRISM. Menu de contexto: Definir como Top Level / Remover Top Level.
- **Testbench** — o banco de testes da simulação (um `_tb.v` Verilog ou um `.py` cocotb). Menu de contexto: Marcar como Testbench / Desmarcar como Testbench.

A barra de status lembra você do que falta: “Sem top-level”, “Sem testbench”. Para um projeto de um único processador, o *testbench* gerado automaticamente pela compilação (`<proc>_tb.v`) costuma bastar — os papéis manuais importam quando você escreve integração própria.

4.5 As três visões da árvore

O botão de alternância no cabeçalho do painel circula entre as visões:

4.5.1 Visão Arquivos

A visão padrão: um *picker* agrupado por processador, mostrando os fontes C++, os Verilog classificados automaticamente (sintetizável vs. *testbench*) e os artefatos relevantes. É a visão de trabalho do dia a dia.

4.5.2 Visão Hierarquia

Depois de uma síntese (por exemplo, ao abrir o PRISM), esta visão mostra a **árvore de instâncias de módulos** do projeto, como o Yosys a enxerga: o top-level no topo, cada instância aninhada abaixo. Só aparece quando há dados de hierarquia.

4.5.3 Visão Pastas

Um explorador de arquivos completo, no estilo do VS Code, com CRUD integral pelo menu de contexto:

- Novo Arquivo... / Nova Pasta... / Novo arquivo Verilog (.v) / Novo testbench cocotb (.py) / Novo .gitignore;
- Recortar, Copiar, Colar, Renomear..., Excluir (com escolha entre Mover para a Lixeira —

restaurável — e Excluir Permanentemente);

- Copiar Caminho / Copiar Caminho Relativo;
- Abrir no Terminal Integrado (abre o TCMD já na pasta) e Revelar no Explorador de Arquivos;
- Atualizar e Recolher Tudo.

Nomes de arquivo são validados como o Windows exige: sem < > : ? *, sem nomes reservados do sistema, sem terminar em ponto ou espaço. Em conflitos de colagem, a AURORA oferece Substituir ou Manter Ambos.

[Figura 6 — print pendente] *As três visões da árvore, lado a lado*

Três prints da mesma região: visão Arquivos, visão Hierarquia (após abrir o PRISM) e visão Pastas com o menu de contexto aberto.

Arrastar e soltar: a árvore aceita a importação de arquivos `.v`, `.sv`, `.vh` e `testbenches cocotb .py` por arrastar-e-soltar. Outros tipos são recusados com uma dica (arquivos `.gtk`, por exemplo, entram pelo seletor próprio da toolbar).

4.6 Manutenção do projeto

- **Backup:** o botão **Backup do projeto** no cabeçalho da árvore gera um `.zip` do projeto inteiro, sob demanda.
- **Arquivos faltantes:** se arquivos registrados no `.spf` sumirem do disco, um aviso lista quantos faltam e oferece dispensá-los (removendo as referências).
- **Fechar projeto:** o botão vermelho no cabeçalho da árvore, com confirmação (“Tem certeza que deseja fechar o projeto atual?”).
- **Renomear projeto:** disponível também via Aurora Intelligence; a operação reescreve o `.spf` e a pasta.

O processador SAPHO

Antes de criar processadores na IDE, vale entender o que exatamente está sendo criado. Este capítulo descreve a arquitetura do *soft-processor* SAPHO do ponto de vista de quem o usa: o que os parâmetros significam, como o processador executa o seu programa e quais são as suas restrições. Não é preciso decorar nada aqui para usar a plataforma — mas quem entende a máquina escreve programas melhores para ela.

5.1 Visão geral da arquitetura

O SAPHO é um processador de **acumulador** com **arquitetura Harvard**:

- **Acumulador**: em vez de um banco de registradores, existe um único registrador central, o acumulador, que recebe o resultado de toda operação da ULA (unidade lógico-aritmética). O padrão dominante é: um operando vem da memória de dados, o outro é o próprio acumulador, e o resultado volta para o acumulador. Uma **pilha de dados** fornece o segundo operando quando a expressão exige.
- **Harvard**: as memórias de programa e de dados são fisicamente separadas, cada uma com sua largura e seu barramento. O programa não pode se sobrescrever, e dados e instruções são acessados em paralelo.

Três traços definem o estilo do processador:

1. **Totalmente parametrizável** — toda largura é um parâmetro Verilog: a palavra de dados (NUBITS), a mantissa (NBMANT) e o expoente (NBEXPO) do ponto flutuante, o operando das instruções, as profundidades das memórias e das pilhas.
2. **Pague pelo que usar** — cada instrução do conjunto é um parâmetro booleano; o montador do YANC liga apenas as instruções que o seu programa realmente usa, e o hardware das demais é eliminado na geração do circuito.
3. **Ciclo curto** — o processador opera com um *pipeline* raso de três estágios (busca, decodificação, execução), sem *forwarding*: a ULA é combinacional e o resultado se acumula a cada ciclo, de modo que nem saltos condicionais quebram o fluxo. O comportamento temporal é simples de prever, o que é uma virtude em instrumentação.

5.2 Os parâmetros fundamentais

Ao criar um processador (Capítulo 6) você escolhe, direta ou indiretamente, os valores abaixo. Eles aparecem como diretivas no topo do arquivo C± e como **parameter** no Verilog gerado.

Parâmetro	Diretiva C±	Significado
NUBITS	#NUBITS	Largura da palavra de dados, em bits. Todo inteiro do programa tem esse tamanho, em complemento de dois.
NBMANT	#NBMANT	Largura da mantissa do ponto flutuante.
NBEXPO	#NBEXPO	Largura do expoente do ponto flutuante.
—	#NDSTAC	Profundidade da pilha de dados.

Parâmetro	Diretiva C±	Significado
—	#SDEPTH	Profundidade da pilha de sub-rotinas (chamadas de função aninhadas).
—	#NUIOIN	Número de portas de entrada.
—	#NUIOOU	Número de portas de saída.
—	#NUGAIN	Constante usada pela função <code>norm()</code> (normalização por divisão fixa).
—	#FFTSIZ	Tamanho da FFT (2^{FFTSIZ} pontos), quando o programa usa os endereçamentos com reversão de bits.

Restrição estrutural do ponto flutuante: a palavra precisa comportar exatamente o formato, isto é, $\text{NUBITS} = \text{NBMANT} + \text{NBEXPO} + 1$ (o bit extra é o sinal). A AURORA valida essa igualdade no formulário de criação de processador e recusa combinações inválidas.

5.3 O ponto flutuante do SAPHO

O SAPHO tem hardware de ponto flutuante **próprio, que não segue o padrão IEEE 754**. O formato é: 1 bit de sinal, NBEXPO bits de expoente em complemento de dois e NBMANT bits de mantissa com o bit líder explícito. Isso permite escolher precisões arbitrárias — uma mantissa de 10 bits com expoente de 5, por exemplo — e economizar hardware exatamente onde a aplicação permite.

Na prática, para o usuário:

- Dentro do programa C±, `float` “simplesmente funciona”: literais, aritmética e as funções da biblioteca operam nesse formato interno.
- A conversão entre o formato IEEE do seu computador e o formato do SAPHO é feita em software pela toolchain (na carga das memórias e na leitura dos resultados), de forma transparente.
- A precisão que você obtém é exatamente a que você configurou: com `#NBMANT 10`, espere cerca de 3 dígitos decimais significativos.

Números **complexos** (tipo `comp` da linguagem) não têm hardware dedicado: cada complexo é um par de *floats* (parte real e imaginária) manipulado componente a componente pelo código gerado.

5.4 Como o programa vira circuito

O processador gerado é composto por poucos módulos Verilog, todos criados a partir de moldes parametrizáveis da pasta de HDL do YANC:

Módulo	Função
<code>processor</code>	O topo: instancia o núcleo e as duas memórias.
<code>mem_instr</code>	Memória de programa (somente leitura), carregada com a imagem <code><proc>_inst.mif</code> .
<code>mem_data</code>	Memória de dados, carregada com a imagem <code><proc>_data.mif</code> ; uma leitura e uma escrita por ciclo.
<code>core</code>	O caminho de dados: acumulador, pilhas, busca de instruções, contador de programa.

Módulo	Função
<code>instr_dec</code>	Decodificador de instruções.
<code>ula</code>	A unidade lógico-aritmética: um submódulo por operação, e só os usados sobrevivem.
<code>addr_dec</code>	Decodificador de endereços para múltiplas portas de E/S.
<code>myFIFO</code>	FIFO genérica, disponível para acoplar o processador ao mundo externo sem depender de IP de fabricante.

O conjunto de instruções usa opcode de 7 bits, com famílias para carga e armazenamento (direto, indireto e com reversão de bits para FFT), pilha, entrada e saída, controle de fluxo (salto incondicional, salto se zero, chamada e retorno de sub-rotina) e cálculo (aritmética inteira e em ponto flutuante, lógica, comparações, deslocamentos e conversões). Você não escreve essas instruções à mão — o compilador $C\pm$ as gera —, mas as verá nas formas de onda: a AURORA exhibe, em *lockstep* com o clock, a instrução assembly e a linha $C\pm$ correspondente a cada ciclo (Capítulo 15).

5.5 Entrada, saída e o mundo externo

O processador conversa com o exterior por portas numeradas de entrada e de saída, criadas conforme `#NUIOIN` e `#NUIOOU`. No programa, elas são acessadas pelas funções `in()`, `out()`, `fin()` e `fout()` (Capítulo 9). No hardware, cada porta vira sinais de dados e de controle no topo do módulo `processor` — é por eles que o seu projeto de FPGA conecta o processador a ADCs, DACs, FIFOs e outros blocos.

Dois recursos opcionais completam a interface: **interrupção** (o processador desvia para um endereço configurado quando o pino de interrupção pulsa) e o marcador `#TOAQUI`, que gera um pino `cheguei` pulsado quando o programa passa por um ponto marcado — útil para sincronizar hardware externo com fases do programa.

5.6 Um processador por tarefa

Como cada processador é gerado sob medida, o padrão de projeto no SAPHO é diferente do mundo dos processadores fixos: em vez de um processador grande que faz tudo, criam-se **vários processadores pequenos**, um por tarefa, conectados entre si por um *top-level* Verilog (por exemplo, um processador que detecta cruzamentos de zero alimentando outro que calcula uma distância). A AURORA suporta esse fluxo diretamente: um projeto pode conter quantos processadores você quiser, cada um com sua pasta, seu `.cmm` e seus parâmetros, todos integrados por um arquivo *top-level* e um *testbench* comuns (Capítulo 4).

Criando processadores: o Hub de Processadores

Com o projeto aberto, o botão **Hub de Processadores** na toolbar abre o formulário que dá vida a um novo processador SAPHO. É aqui que as escolhas de arquitetura do Capítulo 5 viram campos concretos.

[Figura 7 — print pendente] *Modal Criar Hub de Processadores*

O formulário completo, preenchido para o processador `media_movel` do exemplo condutor (Total de Bits 16, Mantissa 10, Expoente 5).

6.1 Os campos do formulário

Campo	Diretiva	Padrão	Validação
Nome do Processador	#PRNAME	procTest_00	Letras, números, _ e -; sem espaços ou acentos.
<i>Seção Formato Numérico</i>			
Total de Bits	#NUBITS	23	Inteiro > 0 e igual a Mantissa + Expoente + 1.
Ganho	#NUGAIN	128	Inteiro > 0 e potência de 2 .
Bits da Mantissa	#NBMAINT	16	Inteiro > 0.
Bits do Expoente	#NBEXPO	6	Inteiro > 0.
<i>Seção Pilhas e Portas</i>			
Tamanho da Pilha de Instruções	#SDEPTH	5	Inteiro > 0.
Tamanho da Pilha de Dados	#NDSTAC	5	Inteiro > 0.
Portas de Entrada	#NUIOIN	1	Inteiro > 0.
Portas de Saída	#NUIOOU	1	Inteiro > 0.

O formulário valida enquanto você digita: um campo inválido ganha borda vermelha e um tooltip explicando a regra (“Total de Bits deve ser igual a Bits da Mantissa + Bits do Expoente + 1”, “Ganho deve ser potência de 2”). O botão **Gerar Processador** só habilita quando tudo está válido. Note que o padrão de fábrica já é consistente: $23 = 16 + 6 + 1$.

Como escolher os valores:

- **Total de Bits** dimensiona os inteiros e o consumo de hardware; 16 bits bastam para muitos sinais de instrumentação, 23 (padrão) dá folga com *float* de boa precisão.
- **Mantissa/Expoente** definem a precisão e a faixa do ponto flutuante (Seção 5.3); se o seu programa só usa inteiros, os valores têm pouco efeito prático, mas a igualdade estrutural continua obrigatória.
- **Pilha de Instruções** limita a profundidade de chamadas de função aninhadas; **Pilha de Dados** limita a complexidade das expressões. Os padrões atendem programas típicos.

- **Portas** de entrada/saída: uma por fluxo de dados que o processador troca com o mundo (Capítulo 9).
- **Ganho**: o divisor fixo da função `norm()`; relevante apenas se você a usa.

6.2 O que é gerado

Ao clicar em **Gerar Processador**, a AURORA cria dentro do projeto:

```
<projeto>/<NomeDoProcessador>/
|-- Software/<NomeDoProcessador>.cmm
|-- Hardware/          (vazia; recebera o Verilog e as memorias)
`-- Simulation/        (vazia; recebera o testbench)
```

O `.cmm` inicial vem com o cabeçalho de diretivas preenchido com os valores do formulário e um `main()` vazio à sua espera:

```
#PRNAME media_movel
#NUBITS 16
#NDSTAC 5
#SDEPTH 5
#NUIOIN 1
#NUIOOU 1
#NBMANT 10
#NBEXPO 5
#NUGAIN 128

void main()
{
    // Ok. Você criou um processador em C++, mas e agora?
}
```

E agora, o Capítulo 7: escrever o programa. O processador também é registrado no `.spf` (nomes duplicados são bloqueados, sem distinção de maiúsculas).

6.3 Configurações de simulação do processador

Ao lado do botão de compilar há a engrenagem **Configurações de simulação do processador**, que abre um popover com três ajustes **por processador**:

Campo	Efeito	Padrão
Clock frequency (MHz)	Frequência do clock do <i>testbench</i> gerado.	100
Number of clocks	Quantos ciclos a simulação executa antes de encerrar.	2000
Show arrays in GTKWave	Emite cada elemento de array como um sinal visível nas ondas (equivale à flag <code>--array</code> do compilador).	desligado

O popover também exibe o **tempo estimado de simulação** (ciclos divididos pelo clock, em μs). Se a sua simulação termina antes de o programa produzir os resultados, aumente o número de ciclos aqui.

6.4 Renomear e excluir processadores

Pelo menu de contexto do processador na árvore. Renomear é uma operação completa: a pasta é movida, os artefatos (`.cmm`, `.asm`, `Hardware/<nome>.v`, `Simulation/<nome>_tb.v`) são renomeados e a diretiva `#PRNAME` dentro do fonte é corrigida — tudo numa ação. As mesmas regras de nome do formulário se aplicam. Excluir remove a pasta do processador e o registro no `.spf`.

O nome do processador e a diretiva `#PRNAME` precisam concordar — é assim que a toolchain conecta o fonte aos artefatos. Renomeie sempre pela AURORA, que mantém os dois lados em sincronia.

Parte III

A linguagem $C\pm$

C±: fundamentos

A C± (lê-se “C mais-menos”; nos arquivos, extensão `.cmm`) é a linguagem principal do SAPHO: um dialeto de C criado pelo NIPS-CERN, enxuto o bastante para virar hardware eficiente e expressivo o bastante para algoritmos de processamento de sinais — incluindo números complexos como tipo nativo e álgebra linear em notação de Dirac. Este capítulo apresenta a estrutura de um programa; os capítulos seguintes cobrem tipos, operadores, E/S, biblioteca e recursos avançados.

Tudo o que está nesta Parte foi extraído da gramática oficial do compilador (`CMMComp.y` e `CMMComp.l` do repositório `nipsclab/yanc`, `toolchain` 5.3). Quando este manual diz que algo “não existe” na linguagem, é porque não existe na gramática — o compilador rejeitará o código.

7.1 A estrutura de um programa

Um programa C± é uma sequência de três tipos de elemento, em qualquer ordem: **diretivas** (linhas iniciadas por `#`), **declarações de variáveis globais** e **funções**. O ponto de entrada é obrigatório: uma função `void main()`. Sem ela, o compilador encerra com erro.

A forma canônica de um programa SAPHO é: diretivas no topo (configurando o processador), depois o `main()` com um laço infinito `while (1)` — afinal, o programa modela um circuito que roda continuamente, processando amostras para sempre:

```
#PRNAME media_movel
#NUBITS 16
#NDSTAC 4
#SDEPTH 2
#NUIOIN 1
#NUIOOU 1
#NBMANT 10
#NBEXPO 5

void main()
{
    int x[4];      // historico das ultimas 4 amostras
    int soma;

    while (1)
    {
        x[3] = x[2];      // desloca o historico
        x[2] = x[1];
        x[1] = x[0];
        x[0] = in(0);      // le nova amostra da porta 0

        soma = x[0] + x[1] + x[2] + x[3];
        out(0, soma >> 2); // media = soma/4, sem usar divisor
    }
}
```

Este é o `media_movel`, o processador que acompanha o manual inteiro: um filtro de média móvel de 4 amostras. Repare em duas escolhas típicas de quem escreve para o SAPHO: o histórico é um array deslocado à mão, e a divisão por 4 é um deslocamento de bits (`>> 2`) — assim o processador gerado não precisa instanciar um divisor, que é um dos blocos mais caros da ULA.

7.2 Diretivas: configurando o processador no próprio código

As diretivas são linhas que começam com **#**, uma por linha, **sem ponto e vírgula**. Elas definem o nome e a arquitetura do processador que será gerado. Quando você cria um processador pela AURORA (Capítulo 6), o formulário preenche essas diretivas por você no `.cmm` inicial; depois disso, o arquivo é a fonte da verdade — editar a diretiva muda o processador na próxima compilação.

Diretiva	Padrão	Significado
#PRNAME <nome>	—	Nome do processador (deve casar com o nome usado no projeto).
#NUBITS <n>	23	Largura da palavra em bits (inteiros em complemento de dois).
#NBMANT <n>	16	Bits de mantissa do ponto flutuante.
#NBEXPO <n>	6	Bits de expoente do ponto flutuante.
#NDSTAC <n>	10	Profundidade da pilha de dados.
#SDEPTH <n>	10	Profundidade da pilha de sub-rotinas (limita chamadas aninhadas).
#NUIOIN <n>	1	Número de portas de entrada.
#NUIOOU <n>	1	Número de portas de saída.
#NUGAIN <n>	64	Divisor fixo usado pela função <code>norm()</code> .
#FFTSIZ <n>	8	Tamanho da FFT (2^n pontos) para o endereçamento bit-reverso.

Lembre da restrição estrutural: $\text{NUBITS} = \text{NBMANT} + \text{NBEXPO} + 1$. No exemplo acima, $16 = 10 + 5 + 1$. A AURORA valida isso ao criar o processador, mas se você editar as diretivas à mão, é sua responsabilidade manter a igualdade.

Existem ainda duas diretivas *comportamentais*, que aparecem no meio do código (não no topo) e são explicadas no Capítulo 10: `#PRACA` (ponto de retorno da interrupção) e `#TOAQUI` (marcador refletido no pino `cheguei`).

7.3 Constantes com #define

A C± tem um pré-processador mínimo embutido: `#define NOME corpo` define uma constante simbólica, expandida onde o nome aparecer.

```
#define SCALE 3
#define OFFSET 7

void main()
{
    int x;
    while (1)
    {
        x = in(0);
        x = x + SCALE;
        out(0, x);
        x = x + OFFSET;
        out(0, x);
    }
}
```

Três limitações importantes em relação ao C tradicional:

- só existem *defines de objeto* — macros com argumentos, como `#define DOBR0(x) ((x)*2)`, não são suportadas;
- não há `#include` nem `#ifdef` no lado C $\pm$ (o fluxo C++, sim, tem pré-processador completo — Capítulo 10);
- o limite é de 256 defines por programa.

O editor da AURORA reconhece os seus `#define` e os `colore` como constantes conforme você digita.

7.4 Comentários

Como em C: `//` comenta até o fim da linha, `/* ... */` comenta blocos.

7.5 Onde o arquivo vive

Dentro de um projeto AURORA, o código de cada processador fica em `<projeto>/<processador>/Software/<processador>.cmm`. A compilação gera o assembly em `Software/`, o Verilog e as imagens de memória em `Hardware/` e o *testbench* em `Simulation/` (Capítulo 12).

C±: tipos, operadores e controle

8.1 Tipos de dados

A C± tem exatamente quatro palavras-chave de tipo:

Tipo	Descrição
<code>void</code>	Ausência de tipo (retorno de funções que não retornam nada).
<code>int</code>	Inteiro de <code>#NUBITS</code> bits, em complemento de dois.
<code>float</code>	Ponto flutuante no formato próprio do SAPHO: mantissa de <code>#NBMANT</code> bits, expoente de <code>#NBEXPO</code> bits (Seção 5.3).
<code>comp</code>	Número complexo: um par de <code>float</code> (parte real e imaginária), tipo de primeira classe da linguagem.

Não existe um tipo “ponto fixo” separado: o papel de fracionário é do `float` configurável. Também não existem `char`, `double`, `struct`, `union`, `enum` nem *strings* (um literal de texto só aparece para inicializar arrays por arquivo, adiante).

8.1.1 Literais complexos e o `i` reservado

Um literal complexo escreve-se `a+bi`, como em matemática:

```
comp z;
z = 3.0 + 4.0i;
```

Por causa disso, o **identificador `i` é reservado** para a unidade imaginária: usar `i` como nome de variável é erro de compilação. Adote `k`, `n`, `idx` para contadores.

8.1.2 Conversões entre tipos

Misturar `int`, `float` e `comp` numa expressão ou atribuição é permitido, mas o compilador emite **aviso** de conversão implícita (por exemplo, ao guardar um `float` num `int`, quando há perda). Usar `float` ou `comp` como condição de `if/while` ou como índice de array também gera aviso. Trate os avisos como cheiro de projeto: conversões explícitas e índices inteiros deixam o hardware gerado mais barato e o comportamento mais óbvio.

8.2 Operadores

Grupo	Operadores
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (módulo, só inteiros) e o <code>-</code> unário
Bit a bit	<code>&</code> <code> </code> <code>^</code> <code>~</code> (inversão)
Deslocamentos	<code><</code> <code>></code> <code>>></code> (deslocamento à direita preservando o sinal)
Relacionais	<code><</code> <code>></code> <code><=</code> <code>>=</code> <code>==</code> <code>!=</code>
Lógicos	<code>&&</code> <code> </code> <code>!</code>

Grupo	Operadores
Incremento	<code>++</code> (como comando ou dentro de expressão, inclusive em elemento de array)

A precedência segue o C. As ausências, porém, importam tanto quanto as presenças:

Não existem em C±: o decremento `--`, os operadores compostos (`+=`, `-=`, `*=`, ...), o operador ternário `?:`, *cast* explícito e **ponteiros** (nem `*` de indireção, nem `&` de endereço — esses símbolos são apenas multiplicação e AND bit a bit). Escreva `x = x - 1;` em vez de `x--;`.

8.2.1 Operadores custam hardware

Cada operador usado pela primeira vez no programa instancia o circuito correspondente na ULA do processador gerado — e o terminal TASM anuncia isso na compilação (“recurso instanciado”). Divisão e módulo são os blocos mais caros; os deslocamentos, os mais baratos. Daí o idioma do exemplo condutor: `soma >> 2` em vez de `soma / 4`. A aritmética *comp* suporta `+` `-` `*` `/`, e cada uma expande para o conjunto de operações reais/imaginárias necessárias.

8.3 Estruturas de controle

Todas as estruturas clássicas do C existem, com a mesma sintaxe:

- `if (cond) { ... } else { ... }`
- `while (cond) { ... }` e `do { ... } while (cond);`
- `for (init; cond; passo) { ... }` — açúcar sintático: o compilador o reescreve como um `while` equivalente;
- `switch (x) { case 1: ... default: ... }` — com *fall-through* real como no C: sem `break`, a execução continua no próximo `case`;
- `break`, `continue` (dentro de laços; fora deles é erro) e `return`.

```
// exemplo: saturador
int satura(int x, int lim)
{
    if (x > lim) return lim;
    if (x < -lim) return -lim;
    return x;
}
```

8.4 Funções

- Sintaxe C: `tipo nome(tipo p1, tipo p2, ...) { ... }`, chamadas com argumentos posicionais, `return` devolvendo o valor.
- O número de argumentos é conferido na chamada (contagem errada é erro de compilação); conversões de tipo em parâmetros e retorno geram aviso.
- **Arrays não podem ser parâmetros de função.** Trabalhe com arrays globais ou dentro do `main()`.
- **Recursão não é suportada no fluxo C±.** Cada variável — incluindo parâmetros e locais — vive num endereço fixo da memória de dados; uma função que chamasse a si mesma sobrescreveria os próprios dados. (É esse endereçamento fixo que permite à AURORA mostrar cada variável por nome nas formas de onda.) Se o seu algoritmo é essencialmente recursivo, use o caminho C++ (Capítulo 10).

- A profundidade de chamadas aninhadas (f chama g, que chama h...) é limitada por `#SDEPTH`, a pilha de sub-rotinas.

```
int add(int a, int b)
{
    return a + b;
}

int triple_sum(int x, int y, int z)
{
    return x + y + z;
}

void main()
{
    while (1)
    {
        int v = in(0);
        out(0, add(v, 1));
        out(0, triple_sum(v, 1, 2));
    }
}
```

8.5 Arrays

Arrays de uma e duas dimensões, com tamanho fixo:

```
int x[128];           // sem inicializacao
int h[8]  "coefs.txt"; // inicializado por arquivo, em tempo de compilacao
int m[8][8] "matriz.txt"; // 2D, tambem por arquivo
```

A inicialização por arquivo lê os valores de um arquivo de texto ao lado do fonte e grava-os diretamente na imagem da memória de dados do processador — os coeficientes do seu filtro já nascem na memória, sem custo de inicialização em tempo de execução.

Duas formas de indexação:

- `x[k]` — indexação normal;
- `x[k]` — **indexação bit-reversa** (repare no fecha-parênteses): o índice é lido com os bits invertidos, o endereçamento clássico da FFT. A quantidade de bits invertidos vem da diretiva `#FFTSIZ`. (Capítulo 10.)

Elementos de array aceitam ++: `hist[k]++;`.

8.6 Variáveis e escopo

Declarações podem aparecer no topo do programa (globais) ou dentro de funções e blocos. Na prática, porém, **todo nome vive num endereço fixo e único** da memória de dados do processador: declarar duas variáveis com o mesmo nome é erro (“a variável já existe”). Pense nas variáveis C± como registradores nomeados do seu circuito — é exatamente isso que elas viram, e é por isso que cada uma aparece por nome na forma de onda da simulação.

C±: entrada, saída e biblioteca padrão

9.1 Entrada e saída

O processador conversa com o mundo por portas numeradas, criadas pelas diretivas `#NUIOIN` (entradas) e `#NUIOOU` (saídas). No código, quatro funções intrínsecas fazem todo o tráfego:

Função	Efeito
<code>x = in(p);</code>	Lê um valor inteiro da porta de entrada <code>p</code> .
<code>x = fin(p);</code>	Lê da porta <code>p</code> convertendo para <code>float</code> .
<code>out(p, expr);</code>	Escreve o valor de <code>expr</code> na porta de saída <code>p</code> .
<code>fout(p, expr);</code>	Escreve na porta <code>p</code> em formato <code>float</code> .

O número da porta é um literal inteiro, validado contra as diretivas: usar `in(2)` num processador com `#NUIOIN 1` é erro de compilação (“não existe porta de entrada”). No hardware, `in()` é uma leitura com *handshake* (o processador sinaliza `req_in` e aguarda o dado), e `out()` pulsa `out_en` com o dado estável no barramento — é o suficiente para acoplar FIFOs, ADCs e DACs.

No nosso `media_movel`, `x[0] = in(0);` lê a amostra e `out(0, soma >> 2);` publica a média: uma porta de cada lado basta.

9.2 Funções especiais

Cinco funções existem para **economizar hardware** — cada uma evita instanciar um bloco caro da ULA ou uma sequência de instruções:

Função	O que faz
<code>norm(x)</code>	Divide <code>x</code> pela constante <code>#NUGAIN</code> (potência de 2). Normalização sem o divisor completo. Só inteiros.
<code>pset(x)</code>	Retorna <code>x</code> se positivo, 0 se negativo — o equivalente de <code>if (x < 0) x=0;</code> numa única instrução.
<code>abs(x)</code>	Valor absoluto. Para <code>comp</code> , retorna a magnitude.
<code>sign(x, y)</code>	Retorna <code>y</code> com o sinal de <code>x</code> .
<code>copy(x, y);</code>	Copia os bits de <code>x</code> em <code>y</code> sem conversão nem checagem de tipo — reinterpretação crua. Use com consciência.

9.3 Funções não-lineares e de arredondamento

Grupo	Funções
Raiz e trigonometria	<code>sqrt(x)</code> , <code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code> , <code>atan(x)</code>
Hiperbólicas	<code>sinh(x)</code> , <code>cosh(x)</code> , <code>tanh(x)</code>

Grupo	Funções
Exponencial e logaritmo	<code>exp(x)</code> , <code>log(x)</code> , <code>pow(x, y)</code>
Arredondamento	<code>floor(x)</code> , <code>ceil(x)</code> , <code>round(x)</code> (retornam float; <code>round</code> arredonda o meio para longe do zero)

Essas funções não são circuitos dedicados: são **macros de assembly** otimizadas (Newton–Raphson para `sqrt`, séries para as trigonométricas) injetadas no seu programa quando usadas. Elas custam instruções e ciclos, não blocos de ULA — exceto pelas operações de float que elas próprias usam.

`pow(x, y)` merece nota: com expoente inteiro constante o compilador gera a sequência mínima de multiplicações; com expoente inteiro variável, um laço em tempo de execução; nos demais casos, a identidade $x^y = e^{y \ln x}$.

9.4 Funções para complexos

Função	Retorno
<code>real(z)</code>	Parte real de <code>z</code> .
<code>imag(z)</code>	Parte imaginária de <code>z</code> .
<code>fase(z)</code>	O ângulo (argumento) de <code>z</code> .
<code>mod2(z)</code>	A magnitude ao quadrado ($a^2 + b^2$ — sem a raiz, mais barato que <code>abs</code>).
<code>complex(re, im)</code>	Monta um <code>comp</code> a partir de dois float.
<code>conj(z)</code>	O conjugado de <code>z</code> .

```
comp x; comp y; comp r;
x = 1.0 + 2.0i;
y = 3.0 + 4.0i;
r = x * y;
fout(0, real(r));
fout(0, imag(r));
```

Nem tudo se aplica a `comp`: `sqrt`, `exp`, `log`, `pow`, `sign` e `pset` sobre complexos são erro de compilação, assim como `z++`. O módulo `%` e `norm()` são exclusivos de inteiros.

9.5 Álgebra linear em notação de Dirac

O recurso mais idiossincrático da C±: operações vetoriais e matriciais escritas em notação *bra-ket*, familiar a quem vem da física. Com `a`, `b` vetores (arrays) e `M` matriz:

Sintaxe	Operação
<code>x = <a b>;</code>	Produto interno de <code>a</code> e <code>b</code> (expressão).
<code>a # 0>;</code>	Zera o vetor <code>a</code> .
<code>a # M b>;</code>	Produto matriz-vetor: $a \leftarrow M b$.
<code>a # c b>;</code>	Vetor escalado: $a \leftarrow c b$.
<code>A # a><b ;</code>	Produto externo: $A \leftarrow a b^T$.

Sintaxe	Operação
<code>out(p, c a>);</code>	Escreve o vetor a escalado por c na porta p .

Essas construções expandem para os laços e instruções indexadas adequadas — são a forma idiomática de escrever filtros FIR, correlações e projeções em $\mathbb{C}^\pm$. Um FIR direto, por exemplo, é uma linha: `y = <h|x>;` com **h** os coeficientes (inicializados por arquivo) e **x** a janela de amostras.

C±: recursos avançados

10.1 Interrupção: #PRACA

O processador SAPHO tem um pino de interrupção (`itr`). Quando ele pulsa, o contador de programa desvia para o ponto do código marcado pela diretiva `#PRACA` — que, diferentemente das diretivas de configuração, aparece **no meio do programa**, marcando a linha de retorno. O uso típico é reiniciar o laço de processamento quando o hardware externo sinaliza um novo evento (um *trigger* de aquisição, por exemplo), sem esperar o programa completar a volta.

10.2 Sincronização com o hardware: #TOAQUI

A diretiva `#TOAQUI`, colocada num ponto do programa, faz o pino `cheguei` do processador pulsar sempre que a execução passa por ali. É um farol para o mundo externo: “cheguei neste ponto do programa”. Serve para sincronizar blocos de hardware com fases do algoritmo — e é também o mecanismo que a AURORA usa internamente: no **teste de hardware** (terminal `THTEST`) e nas simulações com Verilator, a IDE insere um `#TOAQUI` ao final do `main()` para detectar o término do programa e encerrar a simulação no instante certo.

10.3 FFT e o endereçamento bit-reverso

A diretiva `#FFTSIZ n` configura o circuito de reversão de bits para FFTs de 2^n pontos, usado pela indexação `x[k]` (Capítulo 8). Na FFT radix-2, os resultados saem em ordem bit-reversa; com `x[k]` a reordenação é gratuita — o hardware inverte os bits do índice na leitura/escrita, sem instruções extras. O repositório do YANC traz um projeto de exemplo completo (`proc_fft`) com FFT em C±.

10.4 Quanto custa cada construção

O lema “pague pelo que usar” tem uma consequência prática: **o custo em FPGA do seu processador é proporcional ao subconjunto da linguagem que você usa**. Durante a compilação, o terminal TASM informa cada bloco de hardware instanciado à medida que os opcodes aparecem pela primeira vez:

- chamar uma função (`CAL`) instancia a memória de pilha de sub-rotinas;
- usar arrays instancia o circuito de endereçamento indexado; `x[k]`, o de reversão de bits;
- `/` instancia o divisor inteiro; `%`, o módulo; as versões `float`, os blocos de ponto flutuante correspondentes;
- ao final, o TASM resume o percentual da ISA e da ULA efetivamente usados pelo programa.

Regras de bolso para hardware enxuto: prefira deslocamentos a divisões por potências de 2 (ou `norm()`); prefira `mod2()` a `abs()` quando só compara magnitudes; evite `float` se a dinâmica do sinal cabe em inteiros; e lembre que a primeira ocorrência paga o bloco — a segunda ocorrência do mesmo operador é “grátis”.

10.5 O caminho C++

Além da C±, o YANC compila C++ para o mesmo processador, por uma toolchain paralela: o pré-processador CPPPP (com `#include`, `#define` completo e compilação condicional) seguido

do compilador CPPCOMP, que aceita um subconjunto de C99 com extensões de C++ e emite o mesmo assembly intermediário.

O que o caminho C++ oferece que a C± não tem:

- **ponteiros** (para indexar arrays de tamanho fixo; sem alocação dinâmica);
- **recursão**, com quadros de pilha de verdade;
- **structs** simples (POD) e até recursos de C++ como *lambdas* e destrutores (RAII);
- cabeçalhos padrão adaptados: `array`, `cmath`, `cstdint`, `vector` e outros.

O preço: variáveis locais deixam de ter endereço fixo, e com isso **deixam de aparecer por nome nas formas de onda** — o vínculo íntimo entre código e onda, marca do fluxo C±, se perde em parte. Os parâmetros do processador no fluxo C++ têm padrões próprios (palavra de 32 bits, mantissa 23, expoente 8), ajustáveis por `#pragma yanc`.

Recomendação prática: comece todo projeto em C±. Migre para C++ apenas quando precisar especificamente de recursão, ponteiros ou structs — e mantenha os processadores de sinal críticos em C±, onde a visibilidade de simulação é total.

10.6 As mensagens do compilador

Os diagnósticos do YANC são bilíngues — acompanham o idioma da IDE (Capítulo 19) — e têm personalidade: um erro de sintaxe rende “Erro de sintaxe na linha N” com um comentário bem-humorado, e esquecer o `main()` provoca a pergunta clássica sobre onde ele está. Por trás do humor, a informação é séria; as classes principais:

Classe	Exemplos
Estrutura	falta de <code>main()</code> ; <code>break/continue</code> fora de laço; contagem errada de parâmetros.
Nomes	variável inexistente; variável já declarada; uso do <code>i</code> reservado.
Tipos	<code>%/norm()</code> com não-inteiro; intrínsecos não implementados para <code>comp</code> ; <code>++</code> em complexo.
Faixas	estouro de inteiro para o <code>#NUBITS</code> escolhido; <code>float</code> fora da faixa do formato configurado; porta de E/S inexistente.
Avisos	conversões implícitas entre tipos (atribuição, parâmetro, retorno); <code>float/comp</code> em condição ou índice.

Erros aparecem no terminal TCMM com a linha clicável — o clique leva o editor à linha exata (Capítulo 13). A Aurora Intelligence também sabe explicar qualquer mensagem do compilador (ferramenta `lookup_compiler_message`, Capítulo 17).

Parte IV

O editor

O editor

O editor da AURORA é construído sobre o MONACO, o mesmo motor de edição do Visual Studio Code — portanto multi-cursor, minimapa, localizar/substituir (`Ctrl+F`), *undo* ilimitado e a ergonomia geral que você já conhece vêm de fábrica. Este capítulo cobre o que a AURORA acrescenta por cima.

11.1 Abas

Cada arquivo aberto vira uma aba na barra superior do editor:

- o ponto de “modificado” indica alterações não salvas; `Ctrl+S` salva a aba atual, `Ctrl+Shift+S` salva todas;
- as abas podem ser **reordenadas por arraste**;
- `Ctrl+W` fecha a aba atual; `Ctrl+Shift+T` reabre a última fechada;
- `Ctrl+N` cria um arquivo novo.

11.2 Editor dividido

O botão de divisão, que flutua no canto do painel em foco, abre o arquivo atual num novo painel lado a lado — até **três painéis**, cada um com a sua barra de abas. O detalhe importante: painéis que mostram o mesmo arquivo compartilham o *mesmo* documento. Editar de um lado atualiza o outro ao vivo, com *undo* e estado de salvamento unificados. É a forma natural de manter o `.cmm` à esquerda e o `.v` gerado à direita, ou duas regiões distantes do mesmo arquivo.

[Figura 8 — print pendente] *Editor dividido em dois painéis*
O `media\_movel.cmm` à esquerda e o `media\_movel.v` gerado à direita.

11.3 Realce de sintaxe

- **C±**: gramática própria — palavras-chave, diretivas (`#PRNAME`, `#NUBITS...`), funções da biblioteca, literais complexos (sufixo `i`), notação de Dirac. Os `#define` do seu programa são recolhidos dinamicamente e coloridos como constantes.
- **Assembly SAPHO**: gramática própria para os `.asm` gerados.
- **Verilog/SystemVerilog**: gramática do Monaco, enriquecida com *semantic tokens* do tree-sitter.
- **C/C++**: realce via tree-sitter para o caminho C++.

11.4 Diagnósticos e navegação (LSP)

Dois servidores de linguagem trabalham nos arquivos Verilog:

Servidor	O que oferece
VERIBLE	O servidor “completo”: diagnósticos de sintaxe/estilo enquanto digita, formatação, <i>outline</i> de símbolos, <i>hover</i> , ir-à-definição, referências.

Servidor	O que oferece
SLANG	Análise semântica do projeto inteiro (elabora todos os módulos juntos): erros de elaboração, portas incompatíveis, e autocompletar. Pode ser ligado/desligado com <code>Ctrl+Alt+S</code> — útil em projetos grandes.

Os problemas aparecem sublinhados no código e no *hover*. Tudo é *best-effort*: se um servidor não estiver disponível, a edição segue sem ele.

11.5 Formatação automática

`Shift+Alt+F` formata o documento atual (C, C++ e C $\pm$) com o CLANG-FORMAT empacotado. Se o projeto tiver um `.clang-format` próprio, ele é respeitado.

11.6 Busca no projeto

`Ctrl+Shift+F` (ou o botão de busca no cabeçalho da árvore) abre o modal **Buscar nos arquivos**: termo com opções de *diferenciar maiúsculas*, *palavra inteira* e *expressão regular*, resultados agrupados por arquivo, clique leva à linha. A busca ignora `.git` e pastas de artefatos.

11.7 Widget de seleção e IA

Ao selecionar um trecho de código, um pequeno widget (estrela) aparece junto à seleção: ele envia o trecho diretamente para a Aurora Intelligence, já contextualizado — o caminho mais curto para “explique isto” ou “por que isso não compila?” (Capítulo 17).

11.8 Conflitos com o disco

Se um arquivo aberto for alterado por outro programa, a AURORA detecta e pergunta o que fazer no diálogo **Arquivo Modificado Externamente: Manter Versão do Editor, Usar Versão do Disco** ou **Salvar e Recarregar**. Nada é sobrescrito sem a sua decisão.

11.9 Atalhos de teclado

Os atalhos padrão do editor (todos reconfiguráveis nas Configurações, Capítulo 19):

Atalho	Ação
<code>Ctrl+N</code>	Novo arquivo
<code>Ctrl+S</code> / <code>Ctrl+Shift+S</code>	Salvar / Salvar todos
<code>Ctrl+W</code> / <code>Ctrl+Shift+T</code>	Fechar aba / Reabrir última aba
<code>Ctrl+Shift+F</code>	Buscar nos arquivos
<code>Ctrl+Shift+P</code>	Paleta de comandos
<code>Ctrl+Alt+S</code>	Ligar/desligar análise semântica (slang)
<code>Shift+Alt+F</code>	Formatar documento

A posição do cursor (Ln, Col) é exibida na barra de status.

Parte V

Compilação e simulação

Compilação

Com o `media_move1.cmm` escrito, é hora de transformá-lo em hardware. Este capítulo explica o que acontece quando você aciona cada botão de compilação e quais arquivos aparecem no projeto.

12.1 O pipeline do YANC em três estágios

Todo clique em **Compilar C±** executa, por processador, a cadeia de três compiladores do YANC:

1. CMMCOMP — traduz o `<proc>.cmm` em assembly simbólico (`<proc>.asm`), junto com os logs e as tabelas que ligam cada instrução à linha do fonte;
2. APPCOMP — primeira passada sobre o `.asm`: coleta os parâmetros do processador e resolve os endereços de variáveis e rótulos;
3. ASMCOMP — segunda passada: gera o processador em Verilog (`Hardware/<proc>.v`), as duas imagens de memória (`Hardware/<proc>_inst.mif` com o programa, `Hardware/<proc>_data.mif` com os dados) e o *testbench* (`Simulation/<proc>_tb.v`).

O terminal TCMM mostra a saída do primeiro estágio; o TASM, a dos outros dois — incluindo os avisos de “recurso instanciado” que revelam o custo de hardware do seu código (Seção 10.4).

O processador gerado instancia os moldes de HDL do SAPHO (*core*, *ula*, memórias etc.), que acompanham a instalação. O programa **não é um arquivo carregado depois**: as imagens `.mif` nascem embutidas no design — reprogramar significa recompilar e re-sintetizar.

12.2 Os botões de compilação, um a um

Botão	O que faz
Compilar C±	Roda o pipeline completo da Seção 12.1 para os processadores do projeto: do <code>.cmm</code> ao Verilog, memórias e <i>testbench</i> .
Sintetizar Verilog	Verificação dos arquivos Verilog do projeto com o Icarus Verilog: erros de sintaxe e elaboração aparecem no TVERI, com linha clicável.
Analisar Verilog (onda)	O fluxo completo até as formas de onda: compila o que for preciso, roda a simulação e abre o visualizador (Capítulo 14).
Fast Sim	Simulação rápida sem gravação de ondas (requer Verilator selecionado). Para quando só interessam as saídas de texto/arquivo.
PRISM	Sintetiza com o Yosys e abre o diagrama RTL (Capítulo 16). Requer top-level definido.
Teste do processador sintetizado	Constrói um <i>harness</i> C++ (Verilator) em torno do <code><proc>.v</code> e exercita apenas as portas de E/S — o teste de fumaça do processador como caixa-preta. Progresso no THTEST.
Cancelar	Interrompe todos os processos de compilação e simulação em andamento (não fecha a IDE).

O botão **Iniciar Compilação** (raio, na barra de status) é um atalho para o fluxo principal. Todos esses comandos também existem na paleta (`Ctrl+Shift+P`), no grupo *Compile*.

12.3 Pré-requisitos e avisos

- O fluxo de ondas exige um *testbench* (o gerado `<proc>_tb.v` serve); o PRISM exige um **top-level** definido. A barra de status avisa o que falta.
- O processador ativo (mostrado na barra de status) segue o `.cmm` em foco no editor.
- Arquivos são salvos automaticamente antes de compilar.

12.4 Onde cada artefato aparece

Depois de compilar o `media_movel`:

```
media_movel/
|-- Software/
|   |-- media_movel.cmm          (seu fonte)
|   |-- media_movel.asm          (assembly gerado)
|   `-- pc_media_movel_mem.txt   (tabela PC -> linha, p/ ondas)
|-- Hardware/
|   |-- media_movel.v            (o processador em Verilog)
|   |-- media_movel_inst.mif     (imagem da memoria de programa)
|   `-- media_movel_data.mif     (imagem da memoria de dados)
`-- Simulation/
    `-- media_movel_tb.v         (testbench gerado)
```

O `.v` e os `.mif` da pasta `Hardware/` são exatamente o que você leva para o Quartus/Vivado na hora de sintetizar para o FPGA real. O *testbench* gerado produz clock e reset, alimenta o processador com os estímulos de `input_<i>.txt` (um valor por linha, um arquivo por porta de entrada) e grava as saídas em `output_<i>.txt` — além do *dump* de ondas.

Segurança de execução: a AURORA só executa binários da própria toolchain, validados contra uma lista fechada — os botões não passam por um shell, e argumentos críticos são protegidos. É por isso que a IDE não oferece “rodar comando arbitrário” fora do terminal TCMD.

Os terminais

A área inferior da janela reúne seis terminais em abas. Cinco são **consoles de saída** — cada etapa da toolchain escreve no seu, então você sempre sabe onde procurar — e um é um **shell interativo** de verdade.

[Figura 9 — print pendente] *Área de terminais com o TCMM ativo após uma compilação*
Mostrar as seis abas, alguns cartões de mensagem (sucesso/aviso) e os filtros do topo.

13.1 Quem escreve onde

Aba	Nome	Conteúdo
TCMM	Terminal C±	Saída do compilador C± (CMMCOMP): erros, avisos e progresso da tradução para assembly.
TASM	Terminal ASM	Saída do montador (APPCOMP/ASMCOMP): geração do Verilog, memórias, e os avisos de “recurso instanciado”.
TVERI	Terminal Verilog	Saída do Icarus Verilog e da síntese (Yosys/PRISM): diagnósticos de Verilog.
TWAVE	Terminal Wave	Execução da simulação (vvp, Verilator, cocotb) e abertura dos visualizadores de onda.
THTEST	Terminal de teste de hardware	O teste de E/S do processador sintetizado, com barra de progresso própria.
TCMD	Terminal de shell	Um PowerShell interativo completo (Seção 13.4).

Quando você dispara uma ação na toolbar, a AURORA troca automaticamente para a aba certa conforme as etapas avançam.

13.2 Cartões, filtros e o modo verboso

As mensagens dos consoles chegam como **cartões** classificados: **erros**, **avisos**, **sucessos** e dicas/informações — o mesmo código de cores da IDE. No topo da área ficam:

- os **filtros por tipo** com contadores (clique para mostrar/ocultar cada classe); eles atuam quando o **Modo Verboso** está desligado nas Configurações — com o modo ligado (padrão), tudo aparece;
- **Limpar** — esvazia o terminal atual;
- **Exportar log** — salva o conteúdo em arquivo;
- **Recarregar aplicação** — reinicia a interface da IDE.

Cada terminal retém até 5000 entradas.

13.3 Números de linha clicáveis

Erros que citam arquivo e linha — `arquivo.v:42:` do Icarus/Yosys ou “linha 17” do C $\pm$ — viram **links**: o clique abre o arquivo no editor, na linha exata. É o caminho mais rápido do erro à correção.

13.4 TCMD: o shell integrado

A aba **TCMD** é um terminal PowerShell real (xterm), rodando na máquina com um *prompt* próprio da AURORA:

- digite qualquer comando — `git`, `python`, `cd`, scripts do projeto — e tecla `Enter`;
- o diretório atual e as variáveis de ambiente **persistem** entre comandos, como num terminal comum;
- o menu de contexto da árvore de arquivos tem **Abrir no Terminal Integrado**, que abre o TCMD já na pasta escolhida;
- é também o shell que a Aurora Intelligence usa quando você autoriza a ferramenta `run_in_terminal` (Capítulo 17).

Use o TCMD para tarefas auxiliares (Git avançado, scripts Python de análise dos `output_*.txt`, utilitários). Para compilar e simular, prefira os botões da toolbar — eles cuidam de argumentos, caminhos e ordem das etapas por você.

Simulação

Simular é executar o processador gerado, ciclo a ciclo, num simulador de Verilog — vendo cada variável, cada instrução e cada porta no tempo. A AURORA orquestra tudo com um clique em **Analisar Verilog**; este capítulo explica as escolhas disponíveis e como controlá-las.

14.1 Dois motores: Icarus Verilog e Verilator

O seletor de simulador na toolbar alterna o motor global:

	Icarus Verilog (padrão)	Verilator
Como funciona	Interpreta o Verilog (IVERILOG compila, VVP executa).	Converte o design em C++ e compila um executável nativo.
Velocidade	Boa para designs pequenos/médios.	Cerca de 5–10× mais rápido em simulações longas.
Visibilidade	Expõe os sinais internos do processador (variáveis, acumulador, PC...).	Apenas os sinais do topo do <i>testbench</i> .
Uso típico	Depuração: ver o programa rodando por dentro.	Execuções longas e testes de regressão.

Ao alternar, uma notificação resume a troca. Para o dia a dia de depuração do `media_movel`, fique no Icarus; quando quiser rodar milhões de ciclos, mude para o Verilator.

14.2 Dois tipos de testbench: Verilog e cocotb

O *testbench* marca quem estimula o processador:

- **Verilog** (`_tb.v`) — o gerado pela compilação, ou um seu. O gerado cria clock e reset, alimenta cada porta de entrada com o conteúdo de `input_<i>.txt` (um valor por linha) e grava cada porta de saída em `output_<i>.txt`.
- **Python/cocotb** (`.py`) — marque um arquivo Python como *testbench* (menu de contexto da árvore, que também oferece Novo testbench cocotb (`.py`)) e a AURORA roda o *framework* cocotb com o Python embutido, sobre o motor selecionado. Ideal para estímulos complexos, checagens automáticas e comparação com modelos NumPy.

Os dois tipos cruzam com os dois motores: são **quatro caminhos** que convergem no mesmo arquivo de ondas (FST). A simulação roda uma única vez por clique.

14.3 Parâmetros da simulação

No popover **Configurações de simulação do processador** (Seção 6.3): frequência de clock (MHz), número de ciclos e a exibição de arrays nas ondas. O tempo simulado estimado é mostrado ali mesmo. Se a saída do seu programa não aparece, a causa mais comum é simulação curta demais — aumente o número de ciclos.

14.4 Escolhendo os sinais das ondas

Por padrão, o *dump* grava os sinais do escopo do *testbench*. O botão **Configuração de ondas** abre o modal **Sinais para Dump**, onde você escolhe exatamente o que gravar:

- lista de todos os sinais descobertos no projeto, com busca (`Ctrl+F` dentro do modal), opções de maiúsculas/minúsculas e expressão regular;
- **Selecionar tudo, Limpar, Restaurar padrão**;
- filtro Mostrar apenas sinais do processador;
- a opção Surfer: manter janelas abertas (para comparar execuções lado a lado);
- rodapé com o contador de selecionados e **Salvar**.

[Figura 10 — print pendente] Modal Configuração de Ondas

Com alguns sinais do *media_movel* filtrados e selecionados.

Menos sinais = arquivos menores e simulação mais leve — relevante em execuções longas.

14.4.1 Quem manda no dump: a ordem de precedência

Quando a simulação vai gravar ondas, a AURORA decide a lista de sinais nesta ordem:

1. um **.gtkw ativo** no seletor da toolbar — os sinais referenciados nele mandam;
2. a sua seleção no modal **Configuração de ondas**;
3. um `$dumpvars` escrito à mão no seu *testbench* — respeitado, nada é injetado;
4. o padrão: todo o escopo do *testbench*.

O seu *testbench* nunca é modificado: a instrumentação acontece numa cópia temporária.

14.5 O seletor de .gtkw

Ao lado do botão de ondas, o seletor de **.gtkw** escolhe o **layout ativo**: “padrão” usa o layout curado gerado pela AURORA; + Adicionar arquivo .gtkw... registra um layout seu (salvo de dentro do GTKWave). O layout ativo também dirige a precedência do *dump*, acima.

14.6 Rodando

Clique em **Analisar Verilog**. Acompanhe no terminal TWAVE: construção, execução (“Executando simulação VVP...” ou a compilação do Verilator) e a abertura do visualizador escolhido. O Capítulo 15 assume daqui.

Para execuções sem ondas, use **Fast Sim** (Verilator): o programa roda na velocidade máxima e as saídas ficam nos `output_<i>i>.txt`.

14.7 Teste de hardware (THTEST)

O botão **Teste do processador sintetizado** valida o processador como caixa-preta: um *harness* C++ do Verilator alimenta as portas de entrada e confere as saídas, sem *dump* de ondas, com barra de progresso no THTEST. Para detectar o fim do programa, a AURORA usa o mecanismo `#TOAQUI/cheguei` (Seção 10.2): o marcador é inserido automaticamente ao final do `main()` quando necessário.

14.8 Simulando o media_movel

Roteiro completo do exemplo condutor:

1. crie `Simulation/input_0.txt` com uma rampa e um degrau (um inteiro por linha);
2. **Compilar C $\pm$** — confira no TCMM/TASM que tudo passou;
3. confira o motor (Icarus) e o visualizador (GTKWave) nos seletores;
4. **Analisar Verilog** — o GTKWave abre com o layout curado;
5. observe `x[0]...x[3]`, `soma` e a porta de saída suavizando o degrau; a trilha C $\pm$ mostra qual linha do programa executa em cada ciclo;
6. os valores de saída também estão em `output_0.txt`, prontos para conferir num script.

Parte VI

Visualização

Formas de onda: GTKWave e Surfer

Depois que a simulação gera o arquivo de ondas (`.fst` ou `.vcd`), a AURORA abre um visualizador de formas de onda para você inspecionar cada sinal do processador ciclo a ciclo. Há dois visualizadores disponíveis, e a escolha entre eles é uma preferência global da IDE:

- **GTKWAVE** — o visualizador padrão, que já vem empacotado com a AURORA. É um *fork* próprio do laboratório (`nipsclab/gtkwave-nipsclab`) com tema escuro e ajustes de usabilidade.
- **SURFER** — visualizador moderno escrito em Rust, opcional. A AURORA usa o *fork* `surfer-aurora`, mantido pelo grupo no GitLab (`nips-ern`), com decodificadores específicos do SAPHO.

Ambos recebem exatamente a mesma simulação: o formato FST é nativo dos dois, então trocar de visualizador não exige simular de novo.

[Figura 11 — **print pendente**] *Formas de onda de uma simulação do processador `media_movel` abertas no GTKWave*

Mostrar as faixas de clock, instruções (Assembly e C $\pm$ em lockstep), variáveis e portas de E/S.

15.1 O que a AURORA prepara para você

Abrir um `.vcd` bruto num visualizador qualquer significa começar do zero: nenhum sinal selecionado, tudo em binário. A AURORA elimina esse trabalho gerando, a cada simulação, um **layout curado** com os sinais que interessam, já agrupados, coloridos e formatados:

- **Seções por processador:** cada processador do projeto vira um bloco (no Surfer, um grupo colapsável), com divisores nomeados.
- **Sinais de topo:** clock, reset e interrupção primeiro.
- **Portas de E/S** em amarelo, **instruções** em violeta, **variáveis** em laranja, *flags* ao final.
- **Trilha Assembly e trilha C $\pm$:** duas faixas de texto que mostram, a cada ciclo de clock, o mnemônico assembly executado e a linha do seu código C $\pm$ correspondente. As duas avançam em *lockstep* com o clock — é como ver o seu programa rodando dentro do circuito. Elas vêm dos arquivos de tradução `trad_opcode.txt` e `trad_cmm.txt` gerados na compilação.
- **Variáveis complexas:** sinais do tipo `comp` são decodificados para a forma legível “ $a + bi$ ” usando o conversor COMP2GTKW do YANC.

No GTKWave, essa curadoria chega como um arquivo `.gtkw`; no Surfer, como um arquivo de estado `.surf.ron`. Nos dois casos o visualizador já abre pronto para leitura.

15.2 Escolhendo o visualizador

O **seletor de visualizador** fica na toolbar, ao lado do seletor de simulador: **GTKWave** (padrão) ou **Surfer**. A escolha vale para o aplicativo inteiro e é lembrada entre sessões. Se o Surfer estiver selecionado mas o executável não estiver instalado, a AURORA cai automaticamente para o GTKWave, sem erro.

15.3 GTKWave (fork NIPS-CERN)

O GTKWAVE empacotado na AURORA não é o upstream puro: o laboratório mantém o *fork* `nipscernlab/gtkwave-nipscern` com mudanças pensadas para o fluxo SAPHO:

- **Tema escuro por padrão**, com canvases na mesma paleta da AURORA;
- **Zoom-fit automático** ao abrir (a onda inteira cabe na tela) e rótulos justificados à esquerda;
- Remoção de painéis pouco usados e ícones modernizados;
- Zoom animado e suave.

Uso essencial dentro do visualizador:

Ação	Como fazer
Adicionar sinais	Navegue na árvore de escopos (à esquerda), selecione o sinal e use Append /arraste para a área de ondas.
Medir tempo	Clique posiciona o cursor primário; clique com o botão do meio (ou marcador) posiciona o secundário; a barra mostra a diferença.
Zoom	Botões de lupa, Ctrl +roda do mouse, ou zoom-fit para ver tudo.
Mudar formato de um sinal	Botão direito no nome → Data Format (decimal, hexadecimal, binário, com sinal etc.).
Salvar sua seleção	File → Write Save File grava um <code>.gtkw</code> com os sinais, cores e ordem atuais.
Recarregar após nova simulação	File → Reload Waveform .

Se você salvar o seu próprio `.gtkw` no projeto e marcá-lo como ativo na AURORA, ele passa a ter precedência sobre o layout automático: a próxima simulação registra exatamente os sinais que o seu `.gtkw` referencia (Capítulo 14).

15.4 Surfer

O SURFER é um visualizador de ondas moderno (Rust, interface acelerada por GPU) que lê VCD, FST e GHW com carregamento preguiçoso — confortável em simulações grandes. A AURORA baixa o binário do *fork* `surfer-aurora` automaticamente (com verificação de integridade) e o lança como processo externo com o layout curado.

Recursos que valem conhecer:

- **Formatos de exibição ricos**: cada sinal pode ser mostrado como binário, hexadecimal, decimal com/sem sinal, octal, ASCII, ponto fixo, ponto flutuante IEEE (vários tamanhos) e outros decodificadores.
- **Grupos colapsáveis**: no layout da AURORA, cada processador é um grupo que pode ser recolhido — essencial em projetos multi-core.
- **Trilhas Assembly/C $\pm$** através de *mapping translators* instalados automaticamente pela AURORA.
- **Cursors, zoom e navegação rápidos**, com linha de comando interna com autocompletar difuso (tecla de comando dentro do Surfer).
- **Desenho analógico** de sinais, útil para visualizar amostras de sinais processados.

Limitações conhecidas da versão empacotada no Windows: o recarregamento automático do arquivo de ondas não dispara — por isso a AURORA fecha e reabre a janela do Surfer a cada nova simulação.

Os números complexos do SAPHO também funcionam no Surfer: a AURORA pré-decodifica os sinais `comp` com o `COMP2GTKW` e instala o resultado como um *translator*, de modo que a faixa já abre em “ $a + bi$ ”.

PRISM: o visualizador de RTL

O PRISM (*Processor Rendering Interface for Schematic Models*) desenha o seu projeto como um **diagrama de circuito navegável**: módulos, portas e conexões, do top-level até dentro do processador. É a resposta visual à pergunta “que hardware o meu código virou?”.

[Figura 12 — print pendente] *Janela do PRISM com o diagrama do projeto*
O top-level do MeuFiltro renderizado; se possível, um segundo print já dentro do módulo do processador.

16.1 Como abrir

Botão **PRISM** na toolbar (requer um **top-level** definido no projeto — Seção 4.4). A AURORA então:

1. compila o que estiver pendente e verifica a sintaxe dos Verilog;
2. reúne todos os fontes do projeto (HDL do SAPHO, sintetizáveis, top-level, **Hardware/** de cada processador);
3. sintetiza com o YOSYS em modo diagrama (sem mapeamento tecnológico — o desenho preserva a estrutura RTL do seu código);
4. renderiza cada módulo em SVG (via NETLISTSVG) e abre a janela do PRISM.

A saída do Yosys corre no terminal TVERI. A síntese também alimenta a **visão Hierarquia** da árvore de arquivos (Seção 4.5).

16.2 Navegando no diagrama

A janela do PRISM é independente da principal (1400×900, maximizada ao abrir), com o mesmo tema escuro:

- **pan e zoom** com o mouse;
- **clique** num módulo entra nele — o diagrama do submódulo já está materializado, a navegação é instantânea;
- *breadcrumbs* no topo mostram o caminho na hierarquia e permitem voltar;
- **duplo clique** numa célula abre o **código-fonte** correspondente no editor principal, na linha exata da declaração — do desenho ao código num gesto;
- **Recompile** refaz a síntese e o diagrama após mudanças.

Os blocos do processador SAPHO usam *skins* próprias — símbolos desenhados para a ULA, memórias, pilhas — em vez de retângulos genéricos.

16.3 Modo simulação (DigitalJS)

Além do diagrama estático, o PRISM tem um modo **Simular**: o circuito é convertido para o DIGITALJS e vira um esquemático *interativo*, onde você alterna entradas e vê os valores se propagarem pelas conexões, ao vivo. É uma lupa didática para circuitos pequenos:

- o limite é de **3000 células** — acima disso o PRISM recusa e sugere a visão esquemática comum;

- a síntese para este modo tem *timeout* de 45 segundos.

Use o PRISM cedo e com frequência: compilar o `media_movel` e abrir o diagrama depois de cada mudança de código torna palpável a relação entre construções $C_{\pm}$ e custo de hardware — adicione uma divisão ao programa e veja o divisor aparecer no desenho.

Parte VII

Aurora Intelligence

Aurora Intelligence

A AURORA INTELLIGENCE é a assistente de IA integrada à AURORA. Ela conversa sobre o seu projeto — C±, Verilog, arquitetura SAPHO, mensagens do compilador — e, com a sua permissão, **age** sobre a IDE: edita arquivos, compila, dispara simulações, seleciona sinais de onda, opera o Git. Este capítulo cobre a configuração e o uso.

A Aurora Intelligence funciona no modelo *traga a sua chave* (BYOK): você conecta a sua própria conta de um provedor de IA. O provedor e os modelos próprios do NIPS-CERN estão no roteiro do laboratório, mas ainda não foram treinados — até lá, a assistente opera sobre provedores externos.

17.1 Abrindo o painel

Clique no botão **Assistente IA** (estrela) na toolbar. O painel abre à direita, empurrando o editor; a largura é ajustável e lembrada. No cabeçalho: o ícone do provedor ativo, **Chat history** (conversas anteriores), **New chat** e **Close**.

[Figura 13 — print pendente] Painel da Aurora Intelligence com uma conversa Uma pergunta sobre o media_movel respondida, mostrando um cartão de ferramenta executada e o composer embaixo.

17.2 Configurando um provedor

Em **Configurações do Aurora** → aba **Assistente IA**, há um cartão por provedor:

Provedor	Chave	Observações
OpenAI	sk-proj-...	Chave em <i>platform.openai.com</i> .
Anthropic (Claude)	sk-ant-api03-...	Chave em <i>console.anthropic.com</i> .
Google (Gemini)	AIza...	Chave em <i>aistudio.google.com</i> .
DeepSeek	sk-...	Chave em <i>platform.deepseek.com</i> .
Groq	gsk_...	Chave em <i>console.groq.com</i> .
Ollama (local)	—	Sem chave: aponta para o Ollama rodando na sua máquina (http://localhost:11434/v1); o botão Detect encontra a instância e os modelos.

Cada cartão tem os botões **Salvar**, **Testar** (responde “Conectado — N ms”), **Obter uma chave** (abre o console do provedor) e **Remover chave**, além do selo **Configurada/Sem chave** e do campo **Modelo** para escolher um modelo específico (ou manter o padrão do provedor).

Segurança das chaves: as chaves são cifradas pelo cofre de credenciais do Windows (DPAPI) e usadas somente pelo processo principal da AURORA — nunca circulam pela interface nem aparecem em logs. A IDE só exibe *se* há chave, nunca o valor.

17.2.1 CLIs por assinatura

Alternativa às chaves de API: se você assina o CLAUDE CODE (Anthropic) ou o CODEX (OpenAI), a Aurora Intelligence pode usar as CLIs oficiais com o login da sua assinatura. A AURORA baixa a CLI sob demanda (com verificação de integridade), a autenticação é o OAuth da própria

ferramenta — a IDE nunca vê o seu token — e as respostas chegam no mesmo painel de chat. Nesse modo, as ferramentas da AURORA são expostas à CLI por um servidor MCP local e privado.

17.3 O composer

Na caixa de mensagem (“Pergunte à Aurora Intelligence...”):

- seletores de **Provedor** e **Modelo**, e **esforço de raciocínio** (para modelos que suportam);
- **Anexar arquivos ou imagens** — anexos multimodais;
- **Uso** — consumo da conversa;
- **Permissões** — o modo de autorização de ferramentas (abaixo);
- envia; **Parar geração** interrompe.

Selecionar código no editor e clicar na estrela do widget de seleção (Seção 11) envia o trecho já contextualizado.

17.4 Ferramentas: o que a assistente pode fazer

A Aurora Intelligence age por **ferramentas curadas** — as mesmas operações dos botões da IDE, nada além. Elas se dividem em dois níveis:

- **Leitura** (executam sem confirmação): ler o arquivo ativo e os abertos, a árvore do projeto, a saída dos terminais, a lista de processadores, sinais de onda, configurações; consultar a referência embutida de diretivas, opcodes e mensagens do compilador; status/log/diff do Git.
- **Escrita** (pedem confirmação): editar/criar/renomear/excluir arquivos, salvar, abrir arquivos e splits, compilar (`compile_all/compile_step`), cancelar compilação, criar processadores e projetos, mudar configurações, selecionar sinais de onda, abrir o Surfer, rodar comandos no TCMD, e as mutações de Git (`stage, commit, push...`).

Cada ação de escrita aparece como um **cartão de confirmação** no chat — **Permitir** ou **Negar** — antes de executar. O seletor **Permissões** do composer ajusta o rigor: confirmar tudo, confirmar apenas escritas (padrão) ou permitir tudo.

O modo “permitir tudo” deixa a assistente editar arquivos e rodar comandos sem perguntar. Use-o apenas em projetos versionados no Git, onde qualquer alteração é revisável e reversível (Capítulo 18).

17.5 Conversas

Cada conversa é salva localmente; o **Chat history** reabre conversas antigas com contexto completo. Sem conexão, o painel indica “A Aurora Intelligence está offline”.

17.6 Para que ela é boa (no fluxo SAPHO)

- “*Por que este C± não compila?*” — ela lê o terminal, conhece as mensagens do YANC e propõe a correção;
- “*Quanto hardware custa esta função?*” — ela consulta a referência de opcodes e o `.asm` gerado;
- “*Escreva um testbench cocotb que compare com um modelo NumPy*”;
- “*Selecione os sinais das portas e abra as ondas*” — ferramentas de onda por comando de voz... escrito.

Parte VIII

Ferramentas de apoio

Dagr: controle de versão

DAGR (do nórdico antigo “dia”; o painel usa a runa *dagaz* como marca) é o painel de controle de versão da AURORA: uma interface completa de **Git** sobre o repositório do projeto aberto, com integração ao GitHub. Você não precisa instalar nem conhecer a linha de comando do Git para versionar seus projetos SAPHO — embora o TCMD esteja ali para quem preferir.

18.1 Abrindo

Botão **Controle de Versão** na toolbar (ícone de ramificação, com um contador de alterações pendentes) ou o indicador do GitHub na barra de status. Abre o modal **Dagr — Controle de Versão**.

[Figura 14 — **print pendente**] *Modal do Dagr com alterações pendentes*
A aba Changes com arquivos preparados/não preparados, o diff de um deles e a caixa de commit.

Se o projeto ainda não é um repositório, o Dagr oferece inicializá-lo (`git init`) — e criar um `.gitignore` adequado está a um clique no menu de contexto da árvore.

18.2 A aba Changes

O fluxo cotidiano:

1. a lista de **alterações** mostra cada arquivo modificado/criado/excluído; o clique abre o *diff* colorido (adições/remoções) embutido;
2. **prepare** o que vai entrar no commit — por arquivo ou **Preparar tudo** (o inverso, **Despreparar**, também existe);
3. escreva o **Resumo do commit** (e a descrição opcional) e clique em **Commit**.

Apoios: **Amend** (emenda o último commit), **Desfazer último** (desfaz o commit mantendo as alterações), descartar alterações de um arquivo, e avisos para arquivos binários ou *diffs* muito grandes.

A árvore de arquivos também participa: cada arquivo ganha a **decoração de status Git** (modificado, novo, ignorado) na cor correspondente.

18.3 A aba History

O histórico de commits do ramo atual: autor, data, mensagem; o clique mostra os arquivos e o *diff* de cada commit.

18.4 Ramos e stashes

- **Branches**: criar, trocar (com a opção **Guardar e trocar**, que faz *stash* automático das alterações pendentes) e mesclar.
- **Stashes**: a prateleira de alterações guardadas, com **Restaurar** e **Descartar**.

18.5 GitHub: publicar, clonar, sincronizar

- **Entrar com GitHub** — dois métodos: o *device flow* (“Abrimos o GitHub no seu navegador. Digite este código:”) ou um token clássico com escopo **repo** (o Dagr traz um guia embutido de como criar o token). O token é cifrado no cofre do sistema; **Desconectar** o remove.
- **Publicar** — cria o repositório no GitHub a partir do projeto local e envia tudo.
- **Push / Pull / Fetch** — sincronização com o remoto, com progresso ao vivo (o *pull* usa *autostash*: suas alterações locais são guardadas e reaplicadas).
- **Clonar repositórios** — traz um repositório remoto para o disco; se contiver um `.spf`, o Dagr oferece **Abrir projeto no SAPHO**.

Versione desde o primeiro dia. O `.spf` é JSON amigável a *diffs*, os fontes C $\pm$ /Verilog são texto — e um repositório é o pré-requisito para usar a Aurora Intelligence com tranquilidade (todo passo dela vira um *diff* revisável no Dagr).

A Aurora Intelligence expõe o mesmo backend como ferramentas (`git_status`, `git_commit`, `git_push...`), com as mutações sujeitas a confirmação — Capítulo 17.

Configurações

O botão **Configurações do Aurora** na toolbar abre o modal de configurações, organizado em sete seções na navegação lateral. O rodapé traz **Restaurar Padrões** e **Salvar Alterações**.

[Figura 15 — print pendente] *Modal de Configurações*

A seção Geral aberta, mostrando a navegação lateral com as sete seções.

19.1 Geral

- **Habilitar Tooltips** (ligado por padrão) — as dicas ricas ao passar o mouse pelos botões da IDE.
- **Confiar em links externos** (desligado por padrão) — quando ligado, links do chat da Aurora Intelligence abrem no navegador sem confirmação.

19.2 Aparência

Troca do ícone da aplicação: envie uma imagem sua ou **Restaurar ícone padrão**.

A AURORA tem um único tema visual — o tema escuro aurora, cuidadosamente construído — e ele não é configurável. É uma decisão de design da plataforma.

19.3 Idioma

Português (padrão) ou **English**. A escolha muda a interface e as mensagens do compilador YANC (que recebe a flag `-pt/-en` correspondente) — erros de compilação chegam no idioma da IDE.

19.4 Terminal

Modo Verboso (ligado por padrão): mostra todas as mensagens de compilação. Desligue para ativar os filtros por classe (erros, avisos, sucessos, dicas) nos terminais — útil quando só interessam os problemas.

19.5 Atalhos de Teclado

Todos os atalhos da IDE, editáveis: clique num atalho, tecle a nova combinação (esc cancela). Combinações em conflito são recusadas com o aviso “Este atalho já está em uso”. A lista padrão está no Apêndice C.

19.6 Assistente IA

Os cartões de provedores da Aurora Intelligence — chaves, modelos, testes de conexão (Seção 17.2).

19.7 Sobre

A ficha da plataforma: versão instalada, a equipe (desenvolvimento e orientação), os vínculos institucionais (UFJF, NIPS-CERN) e agradecimentos (CAPES, CNPq, FAPEMIG), os links oficiais e o catálogo de **software de terceiros** com as respectivas licenças — do Icarus Verilog ao Monaco, transparência completa sobre o que compõe a AURORA.

Parte IX

Manutenção da plataforma

Atualizações automáticas

O SAPHO evolui rápido, e a AURORA cuida de se manter atual: um atualizador automático embutido verifica, baixa e instala novas versões — sempre com o seu aval.

20.1 Como funciona, do seu ponto de vista

1. **Verificação silenciosa:** cerca de seis segundos após abrir a IDE, a AURORA consulta o canal oficial de *releases* (o repositório `nipscernlab/sapho` no GitHub). Se você já está na última versão, nada acontece — nenhum diálogo.
2. **Oferta:** havendo versão nova, abre a **janela de atualização:** versão nova e atual, o tamanho do download e o *changelog* bilíngue com o que mudou. Nada é baixado sem você pedir.
3. **Download:** ao aceitar, a barra de progresso mostra percentual, MB, velocidade e tempo restante. Você pode continuar trabalhando.
4. **Instalação:** com o download pronto, o botão vira **Reiniciar e instalar**. A AURORA fecha, o instalador roda e a IDE reabre na versão nova.
5. **Confirmação:** na primeira abertura pós-atualização, um *toast* discreto confirma: “O SAPHO foi atualizado para a versão X.Y.Z”.

[Figura 16 — print pendente] *Janela de atualização*

Com uma versão nova disponível: changelog, tamanho do download e o botão de baixar. Se possível, um segundo print com o download em progresso.

A integridade do pacote é verificada (*hash* SHA-512 publicado junto com a *release*) antes de instalar; downloads são incrementais quando possível. Em caso de problema — sem internet, sem espaço em disco, sem permissão — a janela explica em linguagem clara e a IDE continua funcionando na versão atual.

Dois repositórios, por desenho: o desenvolvimento da AURORA acontece no repositório `nipscernlab/aurora`; as versões estáveis são publicadas no repositório `nipscernlab/sapho`, que é o canal que o seu instalador e o atualizador consomem. Você sempre recebe releases estáveis, nunca o estado intermediário do desenvolvimento.

20.2 A toolchain acompanha

Cada instalador carrega, embutida, a versão da toolchain testada com aquela versão da IDE — os compiladores do YANC, os simuladores, o GTKWave, os moldes de HDL. Atualizar a AURORA atualiza o conjunto todo, em versões que foram validadas juntas. Não há nada para você gerenciar manualmente.

20.3 Este manual também se atualiza

O manual que você lê é parte do ecossistema versionado: cada mudança relevante na AURORA, no YANC ou nos componentes reflete numa edição nova. A versão do manual acompanha a da plataforma.

Solução de problemas

Um capítulo de pronto-socorro: os tropeços mais comuns e o caminho de volta.

21.1 Compilação e simulação

Sintoma	Causa e solução
“Erro de sintaxe na linha N” no TCMM	Erro de C $\pm$. Clique na linha para ir ao ponto; confira as ausências da linguagem (sem <code>--</code> , sem <code>+=</code> , sem ponteiros — Capítulo 8).
“yo, where’s the main()?”	Todo programa C $\pm$ precisa de <code>void main()</code> .
Erro citando porta de E/S inexistente	O índice em <code>in()</code> / <code>out()</code> excede <code>#NUI0IN/#NUI0OU</code> . Aumente a diretiva (ou o campo no Hub) e recompile.
Total de bits recusado no Hub	A regra é <code>NUBITS = NBMANT + NBEXPO + 1</code> , e o Ganho deve ser potência de 2.
Botão de ondas não faz nada / aviso na barra de status	Falta <i>testbench</i> (ou top-level, para o PRISM). Compile o C $\pm$ para gerar o <code>_tb.v</code> , ou marque o seu pelo menu de contexto.
A simulação termina antes dos resultados	Poucos ciclos. Aumente Number of clocks no pop-over de configurações do processador.
Simulação lenta demais	Troque o motor para Verilator (5–10 $\times$ mais rápido; só sinais de topo) ou reduza os sinais no Configuração de ondas .
Sinais internos sumiram das ondas	Você está no Verilator, que só expõe o topo do <i>testbench</i> . Volte ao Icarus para depurar por dentro.
Processo travado	Cancelar na toolbar interrompe toda a toolchain sem fechar a IDE.
“Binário Icarus Verilog não encontrado”	A pasta <code>components/</code> da instalação está incompleta ou foi movida. Reinstale o SAPHO pelo instalador oficial.

21.2 Projeto e arquivos

Sintoma	Causa e solução
“Projeto Não Encontrado”	O <code>.spf</code> foi movido ou apagado. Abra pelo novo caminho; o item sai dos recentes sozinho.
Aviso de arquivos faltantes	Arquivos registrados no <code>.spf</code> sumiram do disco. Restaure-os ou dispense o aviso para limpar as referências.

Sintoma	Causa e solução
“Arquivo Modificado Externamente”	Outro programa alterou um arquivo aberto. Escolha entre manter a versão do editor, a do disco, ou salvar e recarregar.
Arrastar arquivo para a árvore não funciona	A importação por arraste aceita <code>.v</code> , <code>.sv</code> , <code>.vh</code> e <code>.py</code> (cocotb). Arquivos <code>.gtkw</code> entram pelo seletor da toolbar.

21.3 Visualizadores

- **Selecionei o Surfer e abriu o GTKWave:** o executável do Surfer não está presente — a AURORA degrada para o GTKWave de propósito. Reinstale/atualize para obter o componente.
- **O Surfer não recarregou a onda nova:** comportamento esperado no Windows — a AURORA fecha e reabre a janela a cada simulação.
- **O PRISM recusou o modo Simular:** o circuito passa de 3000 células; use o diagrama esquemático comum.

21.4 Aurora Intelligence

- **“A Aurora Intelligence está offline”:** sem conexão com o provedor. Verifique a rede e teste a chave na aba Assistente IA (**Testar**).
- **Modelo indisponível:** a AURORA recua automaticamente para o modelo padrão do provedor; escolha outro no seletor do composer.
- **Ollama não detectado:** confirme que o serviço local está rodando e use **Detect** no cartão do Ollama.

21.5 Onde estão os logs

Para relatar um problema, anexe o log principal: `\\APPDATA\\textbackslashAurora-IDE\\textbackslashlogs\\textbackslashmain.log` (o anterior fica em `main.old.log`). O conteúdo dos terminais pode ser salvo com **Exportar log**. Issues são bem-vindas nos repositórios do nipscernlab no GitHub.

Parte X

Apêndices

Referência de diretivas

A.1 Diretivas de configuração

Uma por linha, no topo do `.cmm`, sem ponto e vírgula. “Padrão” é o valor assumido pela toolchain quando a diretiva é omitida; o Hub de Processadores da AURORA escreve todas explicitamente.

Diretiva	Argumento	Padrão	Efeito
#PRNAME	nome	—	Nome do processador; deve casar com o nome no projeto e nos artefatos.
#NUBITS	inteiro	23	Largura da palavra da ULA e dos inteiros (complemento de dois). Deve valer $\text{NBMANT} + \text{NBEXPO} + 1$.
#NBMANT	inteiro	16	Bits de mantissa do ponto flutuante próprio.
#NBEXPO	inteiro	6	Bits de expoente (complemento de dois).
#NDSTAC	inteiro	10	Profundidade da pilha de dados (complexidade de expressões).
#SDEPTH	inteiro	10	Profundidade da pilha de sub-rotinas (chamadas aninhadas).
#NUIOIN	inteiro	1	Número de portas de entrada; <code>in(p)</code> exige $p < \text{NUIOIN}$.
#NUIOOU	inteiro	1	Número de portas de saída; <code>out(p, ...)</code> exige $p < \text{NUIOOU}$.
#NUGAIN	potência de 2	64	Divisor fixo da função <code>norm()</code> .
#FFTSIZ	inteiro	8	Bits invertidos no endereçamento bit-reverso $\mathbf{x}[\mathbf{k}]$ (FFT de 2^n pontos).

Com uma única porta, a ligação é direta; com mais de uma, o hardware instancia um decodificador de endereços automaticamente.

A.2 Diretivas comportamentais

Aparecem no corpo do programa, marcando pontos do código:

Diretiva	Efeito
#PRACA	Marca o ponto para onde o processador desvia quando o pino de interrupção (<code>itr</code>) pulsa.
#TOAQUI	Marca um endereço do programa; o pino <code>cheguei</code> pulsa sempre que a execução passa por ele. Usado pela AURORA para detectar o fim do programa em testes e simulações.

A.3 Pré-processador

Construção	Regras
<code>#define NOME corpo</code>	Constante simbólica (apenas objeto; sem argumentos). Máximo de 256 por programa. Sem <code>#include/#ifdef</code> no fluxo C±.

Referência da biblioteca padrão

Todas as funções intrínsecas da C $\pm$, agrupadas. “Tipos” indica os tipos aceitos; erros de tipo são apontados na compilação.

B.1 Entrada e saída

Função	Tipos	Descrição
<code>in(p)</code>	<code>int</code>	Lê a porta de entrada <code>p</code> .
<code>fin(p)</code>	<code>float</code>	Lê a porta <code>p</code> convertendo para <i>float</i> .
<code>out(p, x);</code>	qualquer	Escreve <code>x</code> na porta de saída <code>p</code> .
<code>fout(p, x);</code>	<code>float</code>	Escreve em formato <i>float</i> .

B.2 Funções especiais (economia de hardware)

Função	Tipos	Descrição
<code>norm(x)</code>	<code>int</code>	$x / \#NUGAIN$ sem instanciar divisor.
<code>pset(x)</code>	<code>int</code> , <code>float</code>	x se $x \geq 0$; senão 0.
<code>abs(x)</code>	todos	Valor absoluto; magnitude para <i>comp</i> .
<code>sign(x, y)</code>	<code>int</code> , <code>float</code>	y com o sinal de x .
<code>copy(x, y);</code>	todos	Cópia bit a bit, sem checagem de tipo.

B.3 Não-lineares (macros de assembly)

Função	Descrição
<code>sqrt(x)</code>	Raiz quadrada (Newton–Raphson). Não aceita <i>comp</i> .
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	Trigonométricas.
<code>atan(x)</code>	Arco-tangente.
<code>sinh(x)</code> , <code>cosh(x)</code> , <code>tanh(x)</code>	Hiperbólicas.
<code>exp(x)</code> , <code>log(x)</code>	Exponencial e logaritmo natural. Não aceitam <i>comp</i> .
<code>pow(x, y)</code>	Potência; estratégia depende do expoente (constante inteiro $\rightarrow$ multiplicações; variável inteiro $\rightarrow$ laço; senão $e^{y \ln x}$).
<code>floor(x)</code> , <code>ceil(x)</code> , <code>round(x)</code>	Arredondamentos (retornam <i>float</i> ; <i>round</i> afasta do zero no meio).

B.4 Complexos

Função	Descrição
<code>real(z)</code> , <code>imag(z)</code>	Partes real e imaginária.
<code>fase(z)</code>	Argumento (ângulo).
<code>mod2(z)</code>	Magnitude ao quadrado.
<code>complex(re, im)</code>	Constrói um <code>comp</code> .
<code>conj(z)</code>	Conjugado.

B.5 Notação de Dirac

Sintaxe	Operação
<code><a b></code>	Produto interno (expressão).
<code>a # 0>;</code>	Zera o vetor.
<code>a # M b>;</code>	Matriz vezes vetor.
<code>a # c b>;</code>	Vetor escalado.
<code>A # a><b ;</code>	Produto externo.
<code>out(p, c a>;</code>	Emite o vetor escalado pela porta p.

B.6 Famílias de instruções do processador

Para leitura das trilhas de assembly nas ondas: os mnemônicos seguem convenções de prefixo/sufixo. `F_` indica a versão em ponto flutuante; `S_`/`SF_`, operando vindo da pilha; `P_`, empilha antes; `_M`, opera sobre memória; `_V`, variante vetorial/indexada.

Família	Mnemônicos principais
Carga/armazenamento	<code>LOD</code> , <code>SET</code> , <code>LDI</code> / <code>STI</code> (indexados), <code>ILI</code> / <code>ISI</code> (bit-reverso)
Pilha	<code>PSH</code> , <code>POP</code>
E/S	<code>INN</code> , <code>F_INN</code> , <code>OUT</code>
Fluxo	<code>JMP</code> , <code>JIZ</code> , <code>CAL</code> , <code>RET</code> , <code>NOP</code>
Aritmética inteira	<code>ADD</code> , <code>MLT</code> , <code>DIV</code> , <code>MOD</code> , <code>NEG</code> , <code>ABS</code> , <code>SGN</code> , <code>PST</code> , <code>NRM</code>
Aritmética float	<code>F_ADD</code> , <code>F_SU1</code> / <code>F_SU2</code> , <code>F_MLT</code> , <code>F_DIV</code> , <code>F_NEG</code> , <code>F_ABS</code> , <code>F_ROT</code> , <code>F_SCL</code> , <code>XPO</code>
Conversões	<code>I2F</code> , <code>F2I</code>
Lógica e bits	<code>AND</code> , <code>ORR</code> , <code>XOR</code> , <code>INV</code> , <code>LAN</code> , <code>LOR</code> , <code>LIN</code>
Comparações	<code>LES</code> , <code>GRE</code> , <code>EQU</code> (e as versões <code>F_</code>)
Deslocamentos	<code>SHL</code> , <code>SHR</code> , <code>SRS</code> (equivale ao operador <code>>>></code>)

Cada mnemônico usado pela primeira vez no programa liga o bloco de hardware correspondente no processador gerado — a lista completa e o percentual de uso da ISA aparecem no terminal TASM a cada compilação.

Atalhos de teclado

Padrões de fábrica; todos reconfiguráveis em **Configurações do Aurora** → Atalhos de Teclado.

Atalho	Ação
<code>Ctrl+N</code>	Novo arquivo
<code>Ctrl+S</code>	Salvar
<code>Ctrl+Shift+S</code>	Salvar todos
<code>Ctrl+W</code>	Fechar aba
<code>Ctrl+Shift+T</code>	Reabrir última aba fechada
<code>Ctrl+Shift+F</code>	Buscar nos arquivos do projeto
<code>Ctrl+Shift+P</code>	Paleta de comandos
<code>Ctrl+Alt+S</code>	Ligar/desligar análise semântica (slang)
<code>Shift+Alt+F</code>	Formatar documento (clang-format)

Dentro do editor valem ainda os atalhos usuais do Monaco/VS Code: `Ctrl+F` localizar, `Ctrl+H` substituir, `Ctrl+Z`/`Ctrl+Y` desfazer/refazer, `Alt+Click` múltiplos cursores, `Ctrl+/` comentar linha, entre outros.

Glossário

SAPHO

Scalable-Architecture Processor for Hardware Optimization: o processador *soft-core* e, por extensão, a plataforma completa (AURORA + YANC + toolchain) e o canal de distribuição.

AURORA

Advanced Utility Running Optimized Resource Architectures: a IDE de desktop da plataforma.

YANC

Yet Another Compiler: a suíte de compiladores que traduz C $\pm$ /C++ em processador Verilog.

C $\pm$ (CMM)

A linguagem de programação da plataforma; dialeto de C com complexos nativos e notação de Dirac. Arquivos `.cmm`.

PRISM

Processor Rendering Interface for Schematic Models: o visualizador de RTL da AURORA.

Aurora Intelligence

A assistente de IA integrada à AURORA.

Dagr

O painel de controle de versão (Git) da AURORA.

NIPS-CERN

O Núcleo de Instrumentação e Processamento de Sinais da UFJF, em colaboração com o CERN — o laboratório que desenvolve a plataforma.

soft-processor / soft-core

Processador descrito em HDL e implementado na malha reconfigurável de um FPGA.

FPGA

Field-Programmable Gate Array: circuito integrado reconfigurável.

Verilog / HDL

Linguagem de descrição de hardware usada pela plataforma.

.spf

SAPHO Project File: o arquivo JSON que define um projeto.

.asm

O assembly intermediário do SAPHO, entre o compilador e o montador.

.mif

Memory Initialization File: imagem de memória (programa ou dados) embutida no design.

testbench

O banco de testes da simulação: um módulo Verilog (`_tb.v`) ou um script Python (cocotb) que estimula o processador.

top-level

O módulo Verilog que integra os processadores e blocos do projeto; topo da síntese.

ULA

Unidade Lógico-Aritmética: o conjunto de operadores do processador; no SAPHO, só os usados são instanciados.

acumulador (ACC)

O registrador central do processador SAPHO, destino de toda operação da ULA.

Harvard

Arquitetura com memórias de programa e dados separadas.

opcode / ISA

Código de operação de uma instrução / o conjunto de instruções do processador.

Icarus Verilog / vvp

O simulador de Verilog padrão (interpreta o design).

Verilator

Simulador de alto desempenho que converte o design em executável C++.

cocotb

Framework Python para escrever *testbenches*.

Yosys

Ferramenta de síntese usada pelo PRISM e pela visão Hierarquia.

GTKWave / Surfer

Os dois visualizadores de formas de onda da plataforma.

VCD / FST

Formatos de arquivo de ondas gerados pela simulação (textual / binário compacto).

.gtkw / .surf.ron

Arquivos de layout de ondas (GTKWave / Surfer): quais sinais, cores e grupos mostrar.

Monaco

O motor de edição de código (o mesmo do VS Code).

LSP

Language Server Protocol: o mecanismo dos diagnósticos e navegação de código (Verible, slang).

BYOK

Bring Your Own Key: o modelo de chaves da Aurora Intelligence.

MCP

Model Context Protocol: como as CLIs de IA acessam as ferramentas da AURORA.

bit-reverso

Endereçamento com bits do índice invertidos ($x[k]$), usado em FFT.

DLP

Dispositivos Lógicos Programáveis: a disciplina da UFJF que usa a plataforma no ensino.